



CS 329Z:Engineering AI Agents

Tool Use

Diyi Yang

Announcements

- Project proposal examples (see Canvas)
- Compute credit
 - Big thanks to Google, Tinker, Laude
 - More to come
- Homework 1 out today
- Proposal due on Oct 9 (Friday)

Lecture Overview

- Wrap up RAG (5 mins)
- Why tools? (5 mins)
- What are tools? (5 mins)
- Learning to call tools: Toolformer (15 mins)
- Scaling to many tools: Gorilla (10 mins)
- Standard interfaces: REST, function calling, MCP (15 mins)
- Evaluating tool use: BFCL, ToolBench, τ -bench (15 mins)

Learning Objective: understand why LLMs need tools, how models learn to call them, how tools are exposed through standard interfaces, and how tool use is evaluated

Issues with RAG

Retrieval-side



Recall ceiling

Missing evidence limits agent

Stale index

Embeddings not update

Permission leak

Use content not allowed to access

Reader-side



Lost in the middle

Position bias

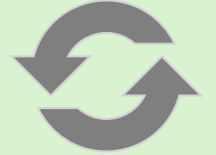
Distraction

Similar but irrelevant content

Evidence conflict

Sources or knowledge conflicts

Loop-side



Compounding error

Early mistakes affect later decision

No stopping rule

Lack clear conditions for finishing

Prompt injection

Retrieved text steers the agent

Problem: How many documents can we use?

Why not get lots of documents and pass it to the LM?

Input Context

Write a high-quality answer for the given question using only the provided search results (some of which might be irrelevant).

Document [1] (Title: Asian Americans in science and technology) Prize in physics for discovery of the subatomic particle J/ψ . Subrahmanyan Chandrasekhar shared...

Document [2] (Title: List of Nobel laureates in Physics) The first Nobel Prize in Physics was awarded in 1901 to Wilhelm Conrad Röntgen, of Germany, who received...

Document [3] (Title: Scientist) and pursued through a unique method, was essentially in place. Ramón y Cajal won the Nobel Prize in 1906 for his remarkable...

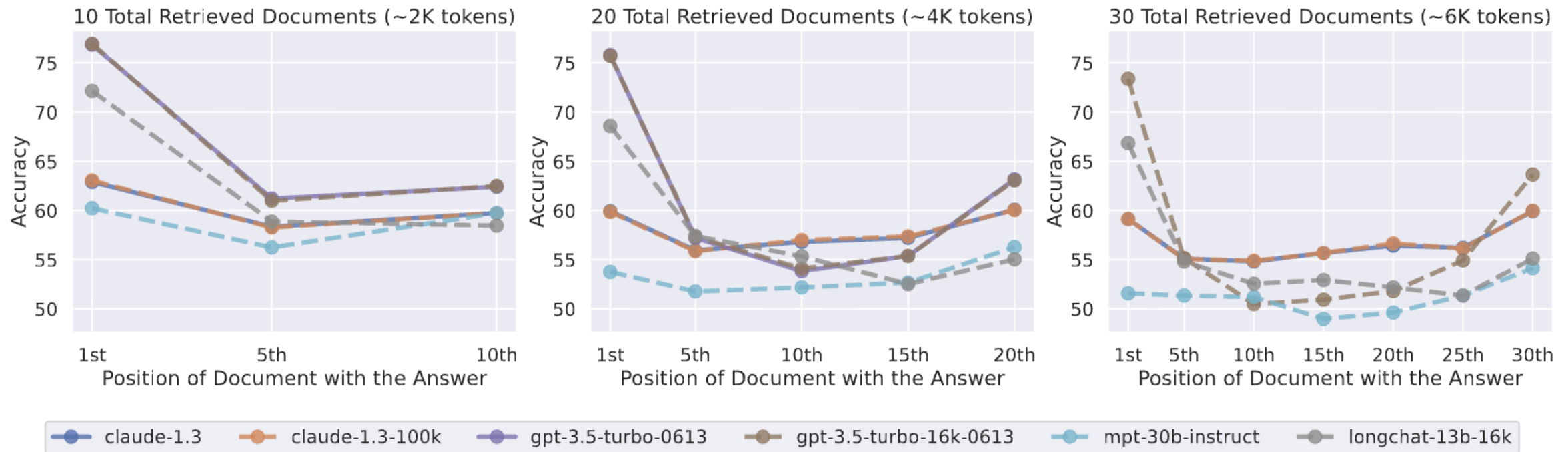
Question: who got the first nobel prize in physics

Answer:

Desired Answer

Wilhelm Conrad Röntgen

LM's can't pay attention to the entire context [\[Liu+ 2023\]](#)

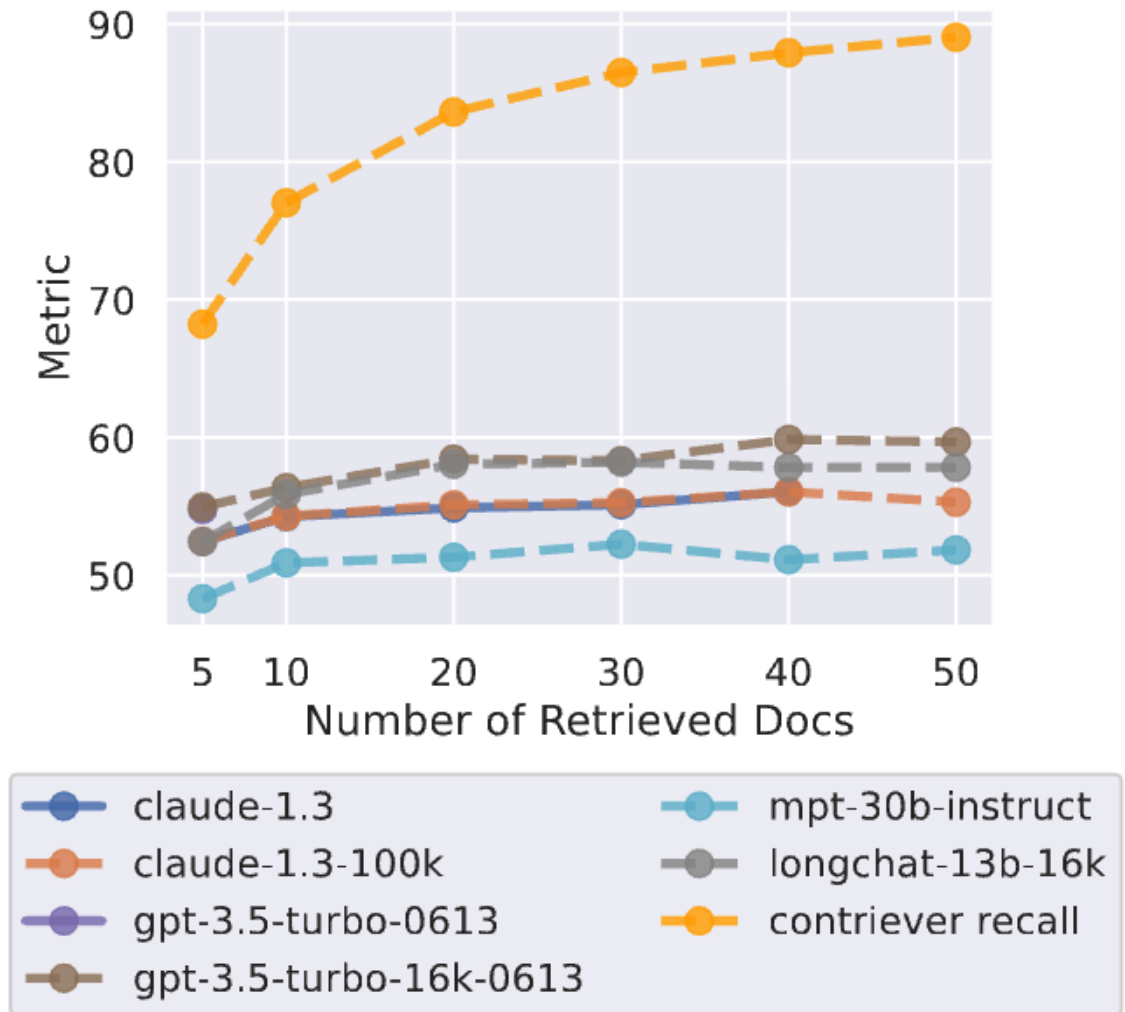


Best Closed-Book performance: GPT-3.5-Turbo, ~56%

Best Oracle (only feed in relevant doc) performance: GPT-3.5-Turbo, ~88.5%

In practice: LMs cannot use many documents

- Retriever performance (yellow) rises slowly to 90% recall
- RAG performance (other lines) saturates very quickly, after 10-20 documents.



LLMs citation and hallucinations

- Unique benefit of RAG: citing the source
- These citations themselves are generated by LLMs
- So the citations could *also be hallucinated* .. How often does that happen?

Search: Create an itinerary for exploring Toronto over a span of two days.

Toronto is a vibrant and diverse city with many attractions and activities to suit different tastes and preferences. Here are some suggestions for creating an itinerary for exploring Toronto over two days:

- **Day 1:** Start your day with a visit to the **CN Tower**, the iconic landmark that offers panoramic views of the city and beyond ¹. You can also experience the thrill of walking on the glass floor or the EdgeWalk, the world's highest hands-free walk ¹. Next, head to the nearby **St. Lawrence Market**, one of the world's best food markets, where you can sample a variety of cuisines and local specialties ². After lunch, take a stroll along **Queen West**, a trendy neighborhood with eclectic shops, galleries, cafes and street art ³. In the evening, enjoy a show at one of the many theaters or comedy clubs in the **Entertainment District**, or catch a game at the **Scotiabank Arena** if you're a sports fan.
- **Day 2:** Explore the history and culture of Toronto at the **Royal Ontario Museum**, one of the largest and most comprehensive museums in North America, featuring exhibits on art, natural history, world cultures and more ⁴. Then, hop on a ferry to the **Toronto Islands**, a group of islands that offer a relaxing escape from the city, with beaches, parks, trails and amusement rides ³ ⁵. You can also rent a bike or kayak to explore the islands at your own pace. For dinner, head to **Chinatown**, one of the largest and most vibrant in North America, where you can find a variety of Asian cuisines and shops ³.

I hope this helps you plan your trip to Toronto. Have fun! 😊

Learn more:

- 1. [cntower.ca](https://www.cntower.ca)
- 2. [travel.usnews.com](https://www.travel.usnews.com)
- 3. [bing.com](https://www.bing.com)
- 4. [rom.on.ca](https://www.rom.on.ca)
- 5. [tripadvisor.com](https://www.tripadvisor.com)

LLMs citation and hallucinations ([Liu et al., 2023](#))

Outputs are easy to read / appear useful to rater (1-5 scale)

- Unique benefit of RAG: citing the source
- These citations themselves are generated by LLMs

<i>Perceived Utility</i> (↑)		<i>Fluency</i> (↑)	
Average Over All Queries		Average Over All Queries	
Bing Chat	4.34	Bing Chat	4.40
NeevaAI	4.48	NeevaAI	4.43
perplexity.ai	4.56	perplexity.ai	4.51
YouChat	4.62	YouChat	4.59
Average	4.50	Average	4.48

But precision and recall are both low..

- So the citations could *also be hallucinated* ..
How often does that happen?

<i>Citation Precision</i> (%; ↑)		<i>Citation Recall</i> (%; ↑)	
Average Over All Queries		Average Over All Queries	
Bing Chat	89.5	Bing Chat	58.7
NeevaAI	72.0	NeevaAI	67.6
perplexity.ai	72.7	perplexity.ai	68.7
YouChat	63.6	YouChat	11.1
Average	74.5	Average	51.5

LLMs citation and hallucinations



INVESTIGATIONS

★ FEATURED

GPTZero finds hallucinations 2025 accepted

GPTZero's analysis 4841 papers accepted at ICLR 2025, at least 100 with confirmed hallucinations



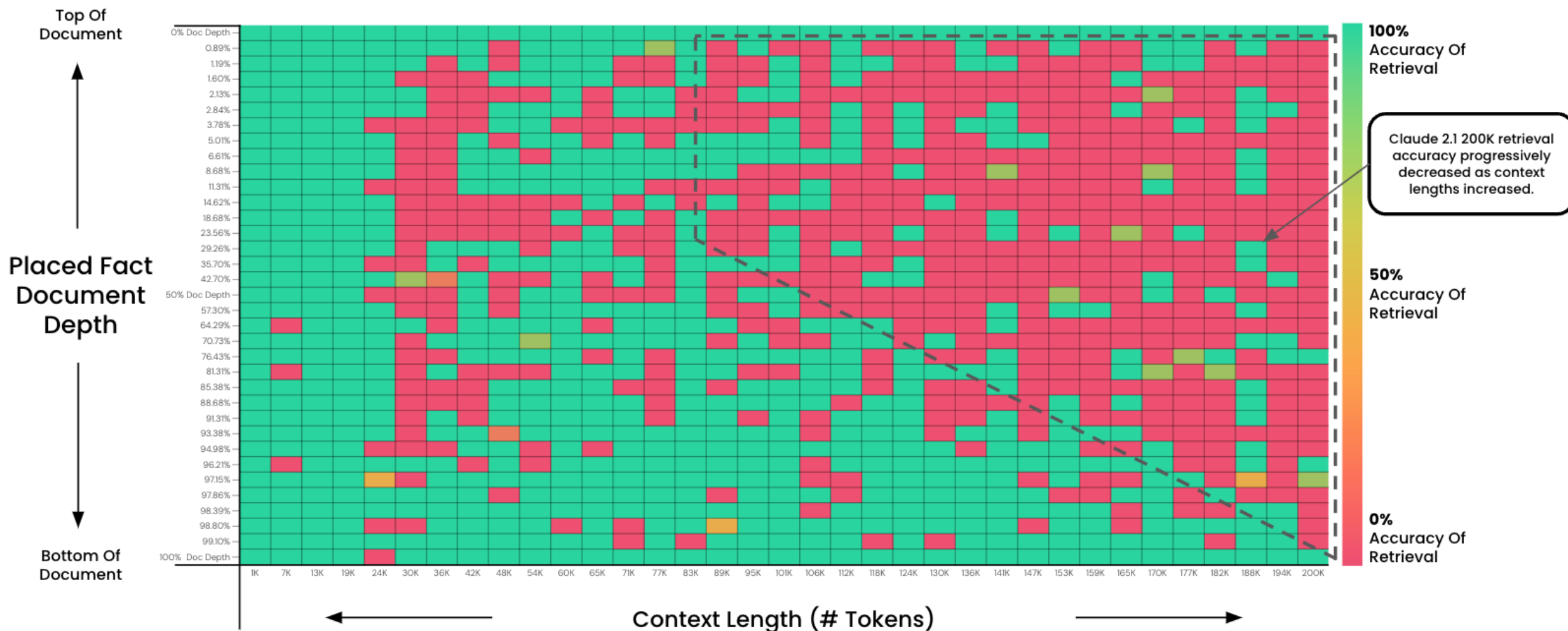
NAZAR SHMATKO, ALEX ADAM, PAUL
JAN 21, 2026 · 26 MIN READ

Taking Actions on Hallucinated References

As in reviews, there are many ways an LLM could be used in the preparation of an ICLR submission, some of which might improve its quality and others that might be harmful. Similar to our approach to reviews, we ran automated LLM content detection on all submissions and flagged submissions with a high proportion of LLM-generated content to ACs. Additionally, we found one widespread issue stemming from LLM use that was relatively straightforward to detect: hallucinated references. Specifically, we found that many submissions included one or more references that referred to documents that didn't exist and/or contained egregiously incorrect bibliographic information. To automatically detect these cases, we made use of a system that automatically extracted references from a given submission and checked them against multiple standard bibliographic databases and a standard web search. This system had a significant false positive rate (for example, it would flag a reference to a paper with a non-English title that a submission's authors had translated to English), so we relied on area chairs to perform a first round of human review of flagged references. Then, we (the program chairs) checked all flagged references manually ourselves. Ultimately, each paper with flagged references was reviewed by at least three humans. Finally, all papers with confirmed hallucinated references were desk rejected, together with an appeal channel to take care of any erroneously flagged papers. This process partially explains the relatively high desk rejection rate at this year's ICLR compared to past years.

Pressure Testing Claude-2.1 200K via "Needle In A HayStack"

Asking Claude 2.1 To Do Fact Retrieval Across Context Lengths & Document Depth

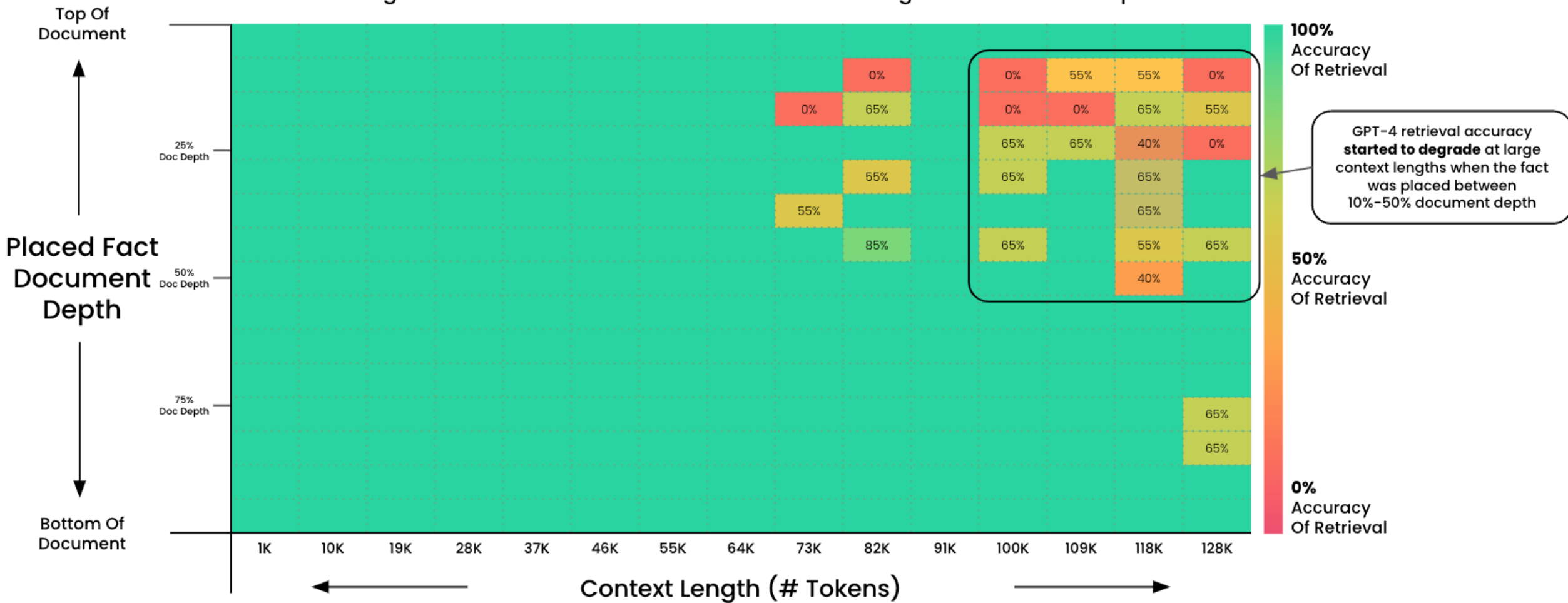


Goal: Test Claude 2.1 Ability To Retrieve Information From Large Context Windows

A fact was placed within a document. Claude 2.1 (200K) was then asked to retrieve it. The output was evaluated (with GPT-4) for accuracy. This test was run at 35 different document depths (top > bottom) and 35 different context lengths (1K > 200K tokens). Document Depths followed a sigmoid distribution

Pressure Testing GPT-4 128K via "Needle In A HayStack"

Asking GPT-4 To Do Fact Retrieval Across Context Lengths & Document Depth



Goal: Test GPT-4 Ability To Retrieve Information From Large Context Windows

A fact was placed within a document. GPT-4 (1106-preview) was then asked to retrieve it. The output was evaluated for accuracy.

This test was run at 15 different document depths (top > bottom) and 15 different context lengths (1K > 128K tokens).

2x tests were run for larger contexts for a larger sample size.

Issues with RAG

Retrieval-side



Recall ceiling

Missing evidence limits agent

Stale index

Embeddings not update

Permission leak

Use content not allowed to access

Reader-side



Lost in the middle

Position bias

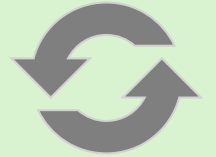
Distraction

Similar but irrelevant content

Evidence conflict

Sources or knowledge conflicts

Loop-side



Compounding error

Early mistakes affect later decision

No stopping rule

Lack clear conditions for finishing

Prompt injection

Retrieved text steers the agent

Lecture Overview

- Wrap up RAG (5 mins)
- Why tools? (5 mins)
- What are tools? (5 mins)
- Learning to call tools: Toolformer (15 mins)
- Scaling to many tools: Gorilla (10 mins)
- Standard interfaces: REST, function calling, MCP (15 mins)
- Evaluating tool use: BFCL, ToolBench, τ -bench (15 mins)

LLMs Cannot Solving That?

Who is the current President of the United States?

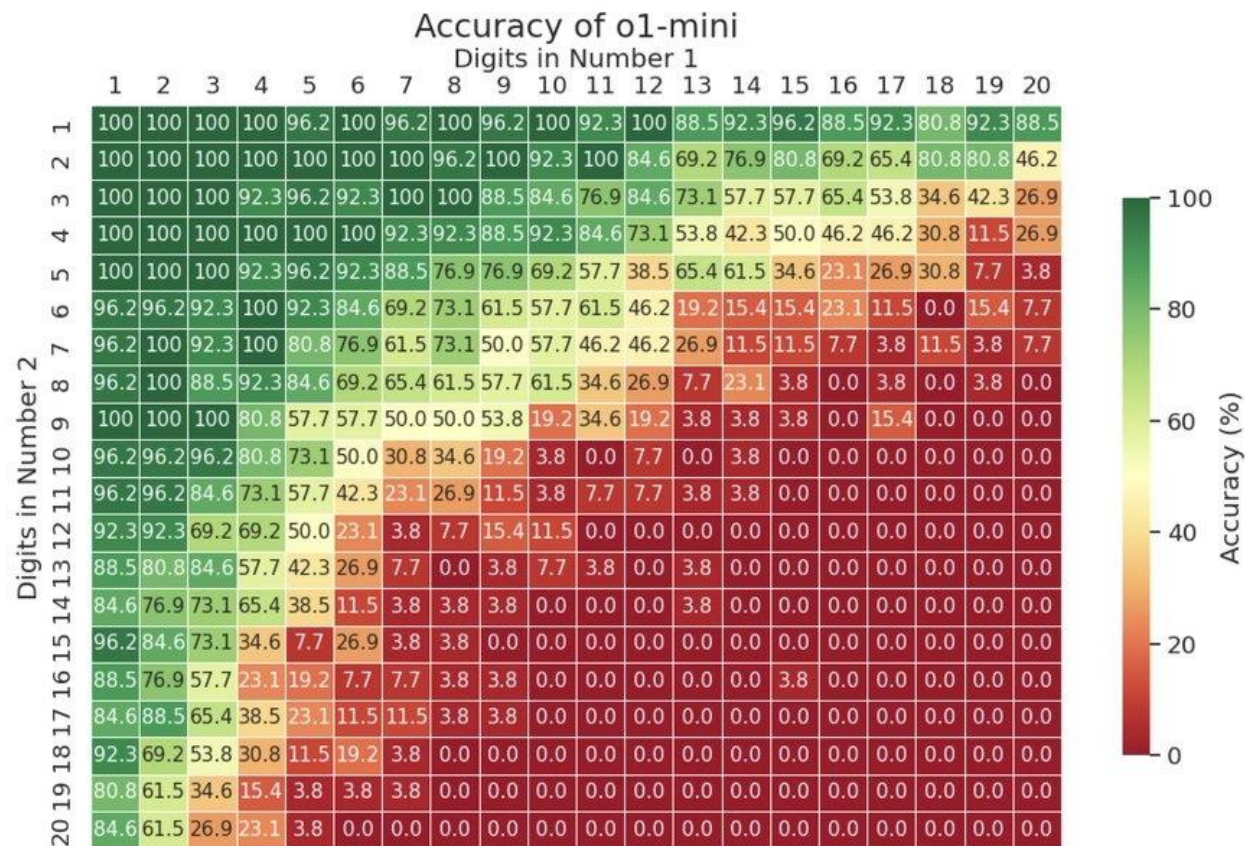
I don't have real-time data or i

Does sparks of AGI (i.e., GPT4) solve

Multiply 999 by 867.

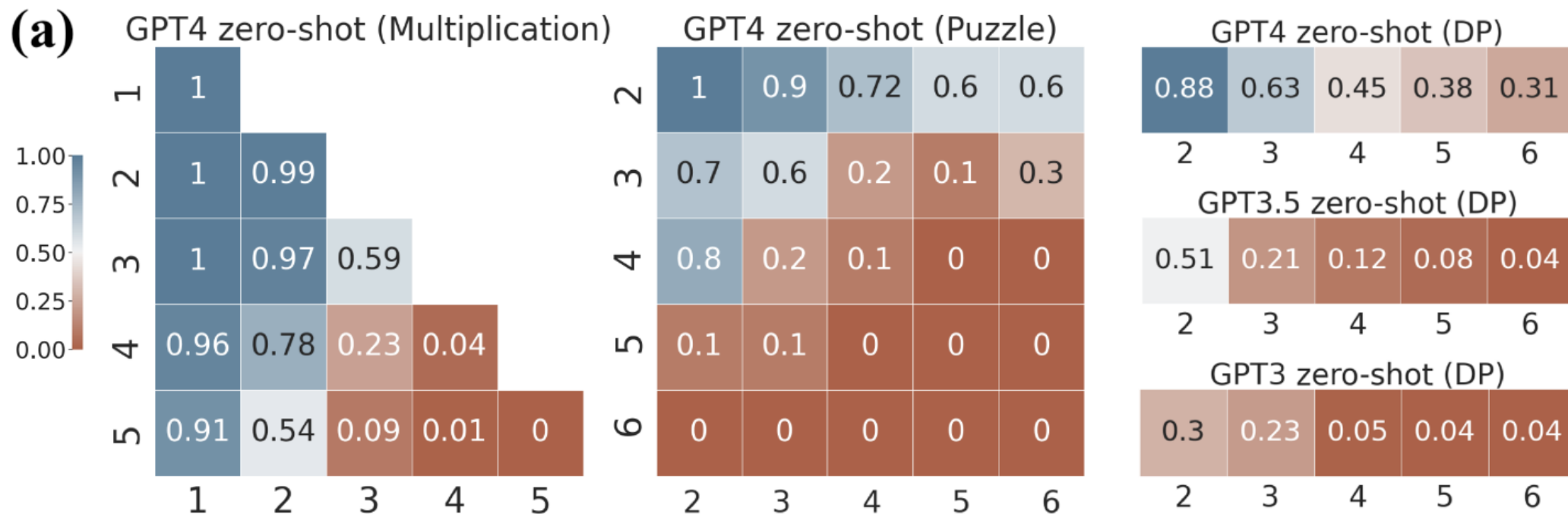
The product of multiplying 999 by 867 is

GPT4 achieves only **59% accuracy** on multiplication.



<https://x.com/yuntiandeng/status/1836114401213989366>

LLMs Struggle as Task Complexity Increases



Zero-shot accuracy. Axes refer to problem sizes (number of digits in multiplication, number of houses and attributes in puzzle, and sequence length in the DP task) ([Dziri et al., 2023](#))

Why Tools?

What LLMs can't compute

1. Arithmetic at scale
2. Multi-step logical deduction
3. Exact, verifiable operations

What LLMs never saw in training

1. Long-tail facts
2. Post-cutoff events
3. Private and proprietary data

What LLMs can't reach

1. Non-language interfaces (e.g., APIs, databases)
2. The physical world
3. Live state (e.g., time, prices, sensors)

What Agents can do

- Calculator, code interpreter
- Symbolic solver, verifier
- Deterministic execution

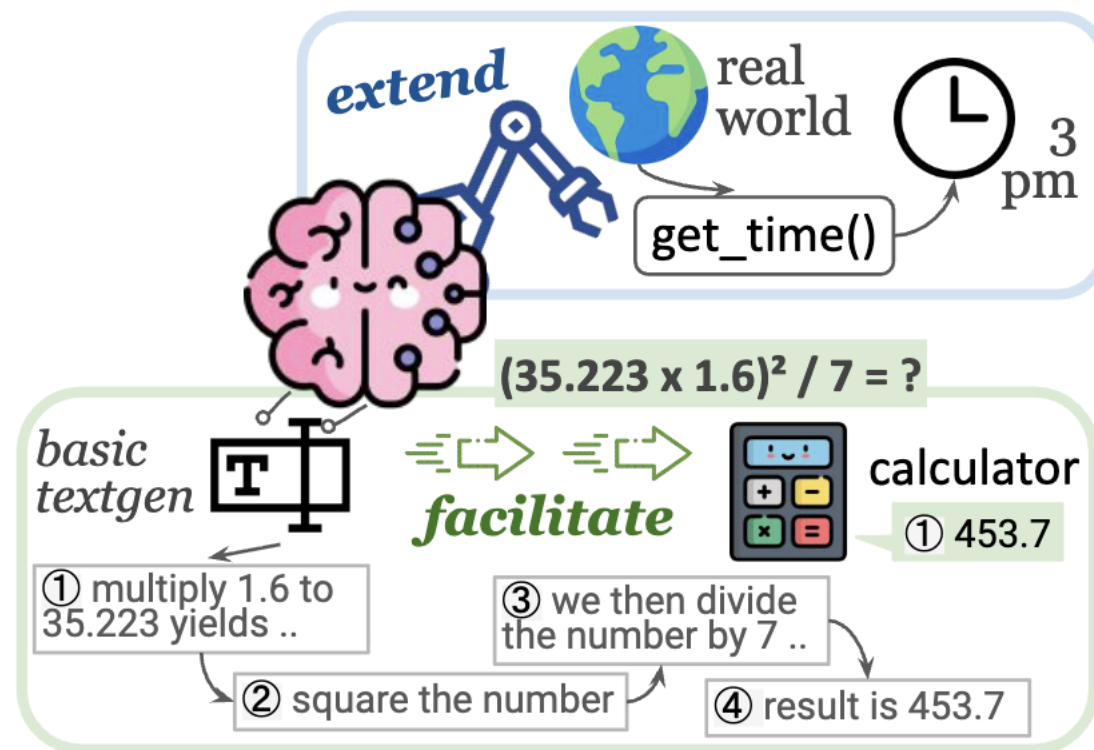
- Web search, retrieval
- Live APIs, news feeds
- RAG over internal corpora

- Function calling, MCP
- Browsers, robots
- Clocks, sensors, market data



What Are Tools?

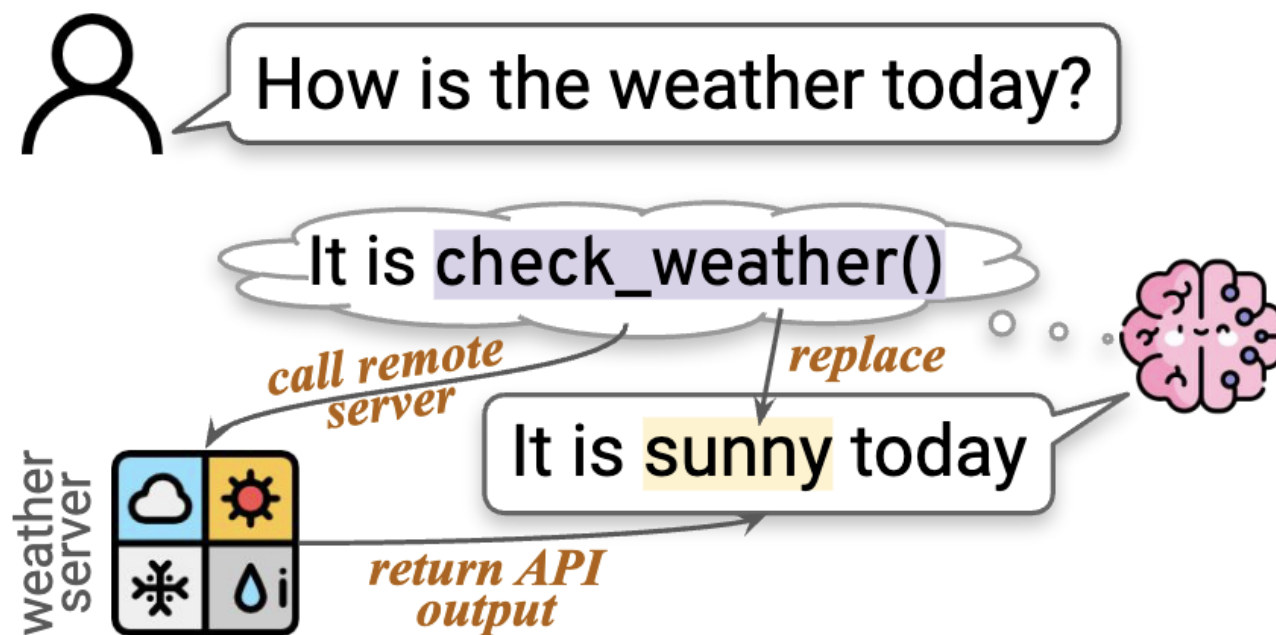
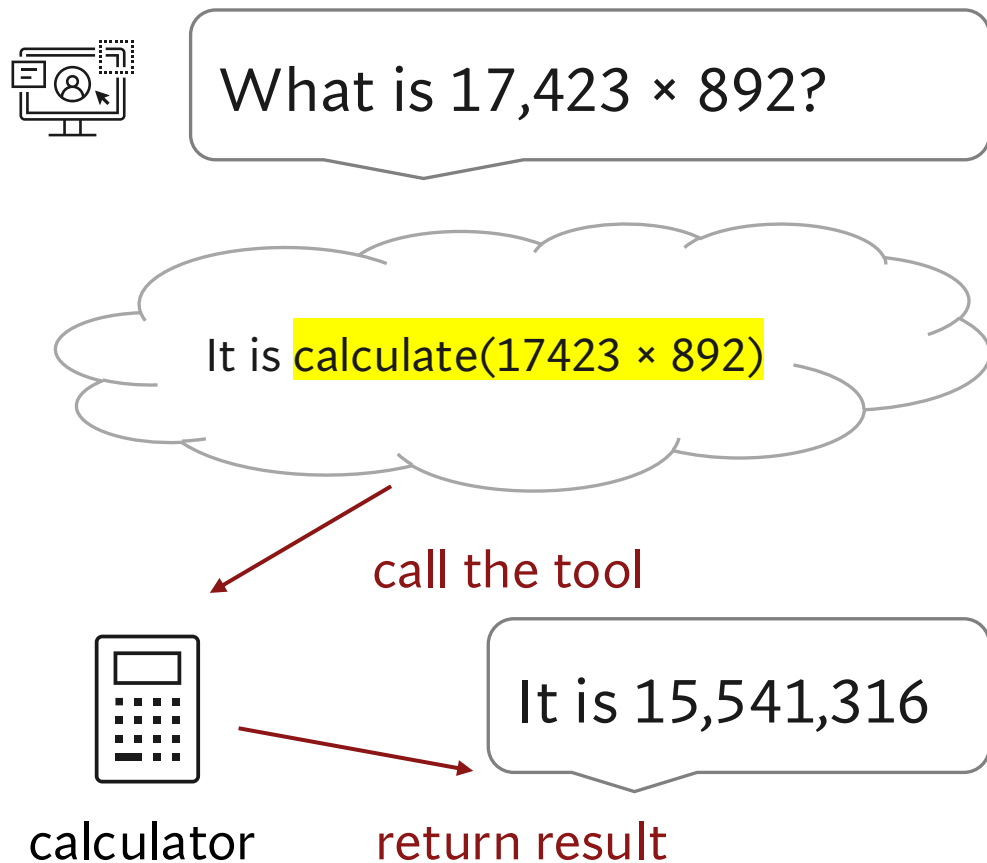
An LM-used tool is a **function interface** to a computer program that runs **externally** to the LM, where the LM generates the function calls and input arguments in order to use the tool



Scenarios of Tool Use

Category	Example Tools
 Knowledge access	<code>sql_executor(query: str) -> answer: any</code> <code>search_engine(query: str) -> document: str</code> <code>retriever(query: str) -> document: str</code>
 Computation activities	<code>calculator(formula: str) -> value: int float</code> <code>python_interpreter(program: str) -> result: any</code> <code>worksheet.insert_row(row: list, index: int) -> None</code>
 Interaction w/ the world	<code>get_weather(city_name: str) -> weather: str</code> <code>get_location(ip: str) -> location: str</code> <code>calendar.fetch_events(date: str) -> events: list</code> <code>email.verify(address: str) -> result: bool</code>
 Non-textual modalities	<code>cat_image.delete(image_id: str) -> None</code> <code>spotify.play_music(name: str) -> None</code> <code>visual_qa(query: str, image: Image) -> answer: str</code>
 Special-skilled LMs	<code>QA(question: str) -> answer: str</code> <code>translation(text: str, language: str) -> text: str</code>

Basic Tool Use Examples



Lecture Overview

- Wrap up RAG (5 mins)
- Why tools? (5 mins)
- What are tools? (5 mins)
- Learning to call tools: Toolformer (15 mins)
- Scaling to many tools: Gorilla (10 mins)
- Standard interfaces: REST, function calling, MCP (15 mins)
- Evaluating tool use: BFCL, ToolBench, τ -bench (15 mins)

LLMs can teach **themselves** to use tools [\(Schick et al., 2023\)](#)

- Toolformer trains LMs to decide which APIs to call, when to call them, what arguments to pass, and how to best incorporate the results.
- Use self-supervised learning with a handful of demonstrations for each API.

The New England Journal of Medicine is a registered trademark of **[QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society]** the MMS.

Out of 1400 participants, 400 (or **[Calculator(400 / 1400) → 0.29]** 29%) passed the test.

The name derives from "la tortuga", the Spanish word for **[MT("tortuga") → turtle]** turtle.

The Brown Act is California's law **[WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.]** that requires legislative bodies, like city councils, to hold their meetings open to the public.

Toolformer: Candidate Tools

- Q&A system, calculator, search engine, translation system, calendar.

What other name is
Pittsburgh known by?



The Steel City

War memorial
Flodden



[...] was created in
memory
of the Battle of
Flodden.

$3435 * 235 / 9$



89691.67

∅



Thursday, March
10, 2019

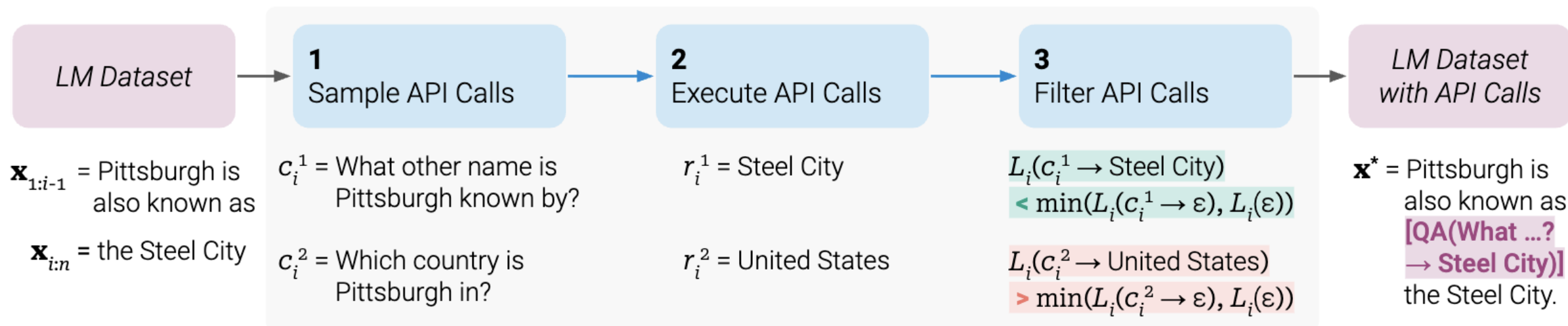
Os Melhores
Escolas em Jersey



The Best Schools
in Jersey

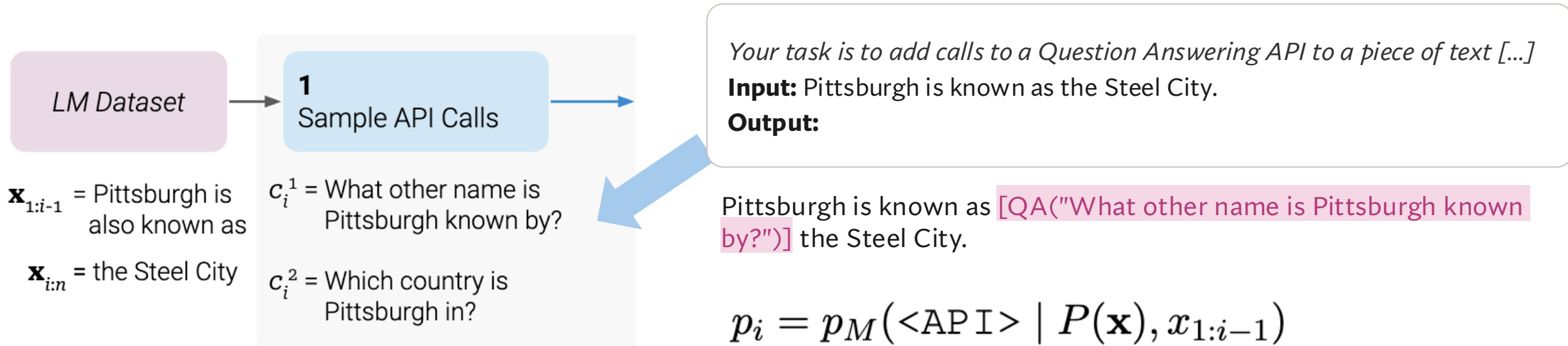
Toolformer: Method

- Create a new dataset augmented with API calls
- Finetune GPT-J (6B) using this new dataset



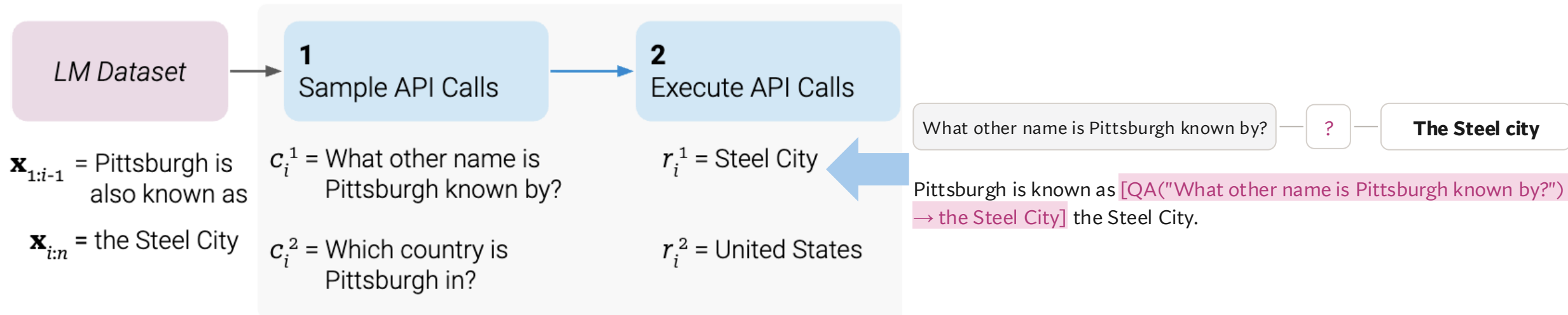
Toolformer: (1) Sample API Calls

For an input $\mathbf{x}$, select positions i where $p_i > \tau_s$, then sample API call candidates $c_i^1, c_i^2, \dots, c_i^m$.



Toolformer: (2) Execute API Calls

We then execute these API calls and filter out all calls which do not reduce the loss L_i over the next tokens.



Toolformer: (3) Filter API Calls

Weighted cross-entropy loss
$$L_i(\mathbf{z}) = - \sum_{j=i}^n w_{j-i} \cdot \log p_M(x_j \mid \mathbf{z}, x_{1:j-1})$$

Compare two differences of this loss:

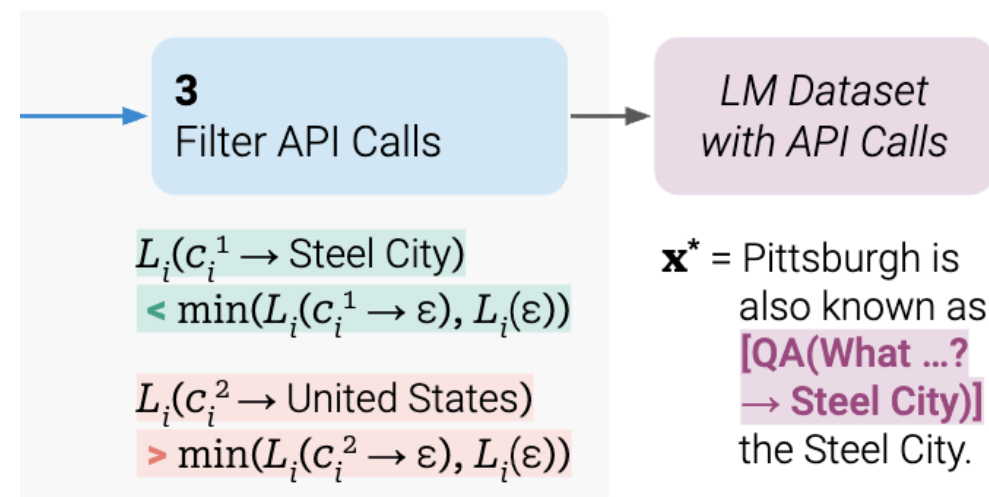
$$L_i^+ = L_i(\mathbf{e}(c_i, r_i)) \quad \text{Both API call and execution output}$$

$$L_i^- = \min(L_i(\varepsilon), L_i(\mathbf{e}(c_i, \varepsilon)))$$

Doing no API call and doing an API call without output

Keep API calls larger than a threshold τ_f

$$L_i^- - L_i^+ \geq \tau_f$$



Toolformer: (3) Filter API Calls - Example

$L_{\bullet}(\text{PREFIX}) = -\log p(\text{the Steel City.} \mid \text{PREFIX})$

A. No API Call $(\text{Pittsburgh is known as}) = 2.5$

B. Non-executed API Call $L_B(\text{Pittsburgh is known as [QA("What other name is Pittsburgh known by?")} \rightarrow ?]) = 2.1$

C. Executed API Call $L_C(\text{Pittsburgh is known as [QA("What other name is Pittsburgh known by?")} \rightarrow \text{Steel City}]) = 0.8$

$$\text{Usefulness} = \min(L_A, L_B) - L_C = \min(2.5, 2.1) - 0.8 = 1.3$$

3
Filter API Calls

$L_i(c_i^1 \rightarrow \text{Steel City})$
 $< \min(L_i(c_i^1 \rightarrow \epsilon), L_i(\epsilon))$

$L_i(c_i^2 \rightarrow \text{United States})$
 $> \min(L_i(c_i^2 \rightarrow \epsilon), L_i(\epsilon))$

LM Dataset
with API Calls

$\mathbf{x}^*$ = Pittsburgh is
also known as
[QA(What ...?
→ Steel City)]
the Steel City.

Example Text and Usefulness Scores

2.11

The WL will be open on Friday, [Calendar() → Today is Thursday, March 9, 2017.] March 10, and Sunday, March 19 for regular hours.

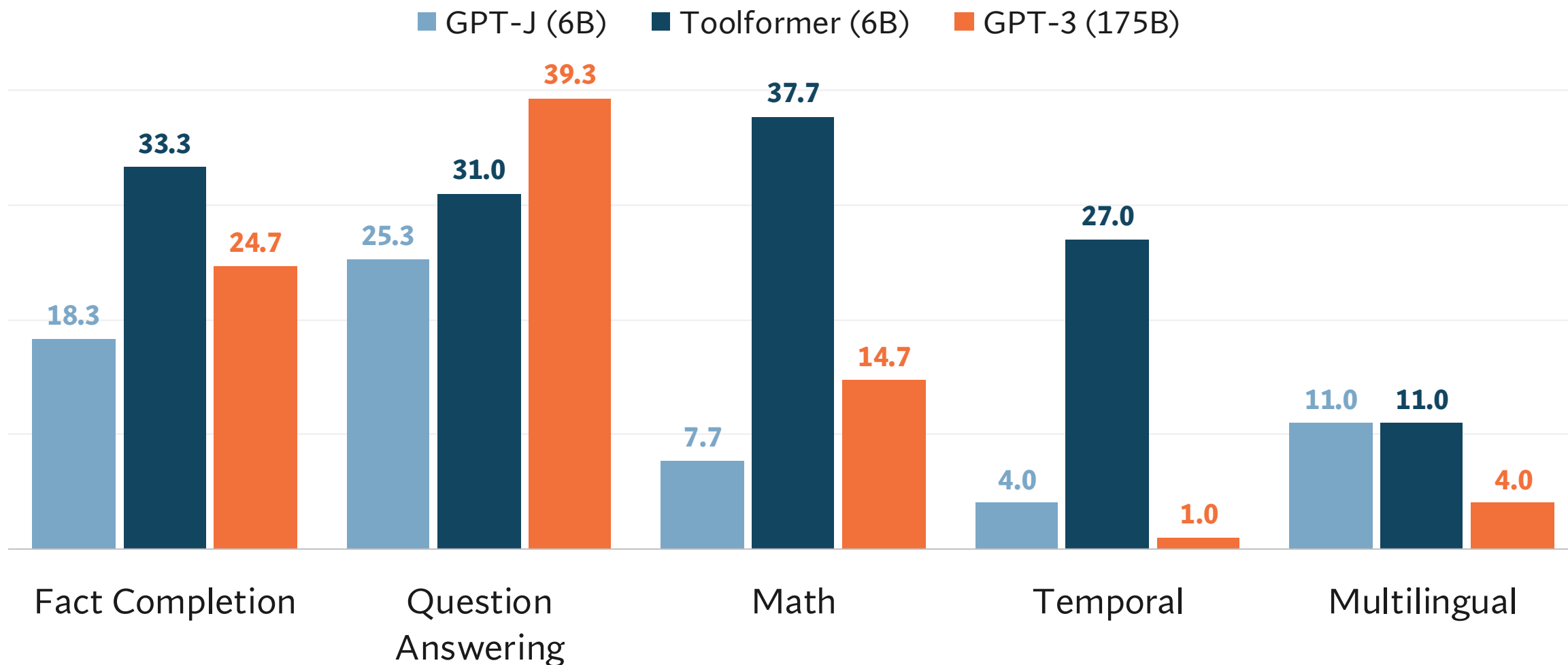
1.59

If Venus had an atmosphere similar to Earth's, then you would expect Venus' main temperature to be 499 K rather than 735 K which is [Calculator(735 / 499) → 1.47] 1.47 times hotter.

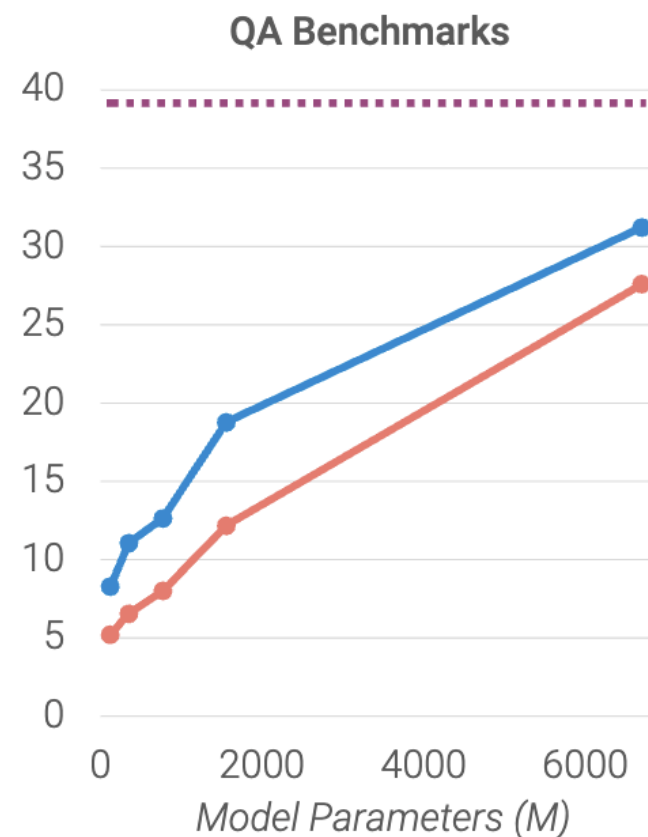
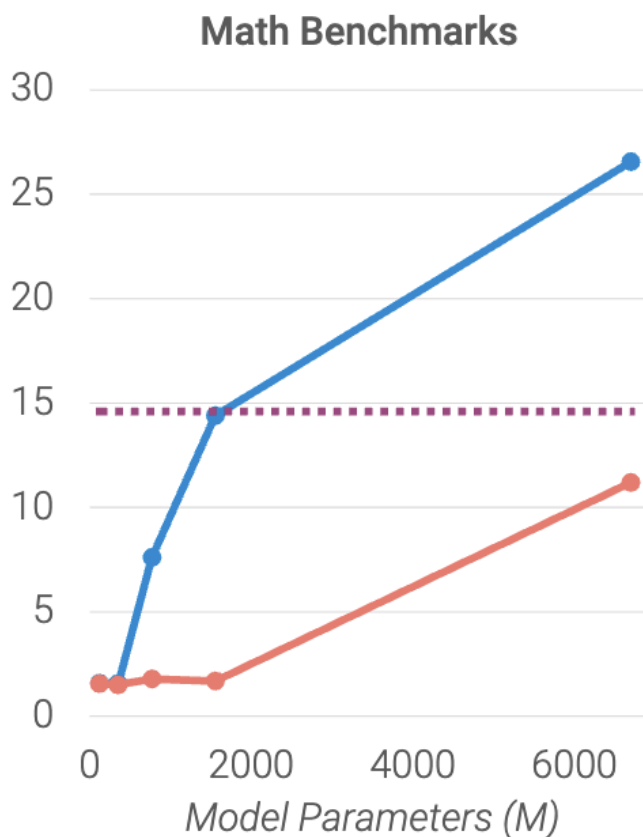
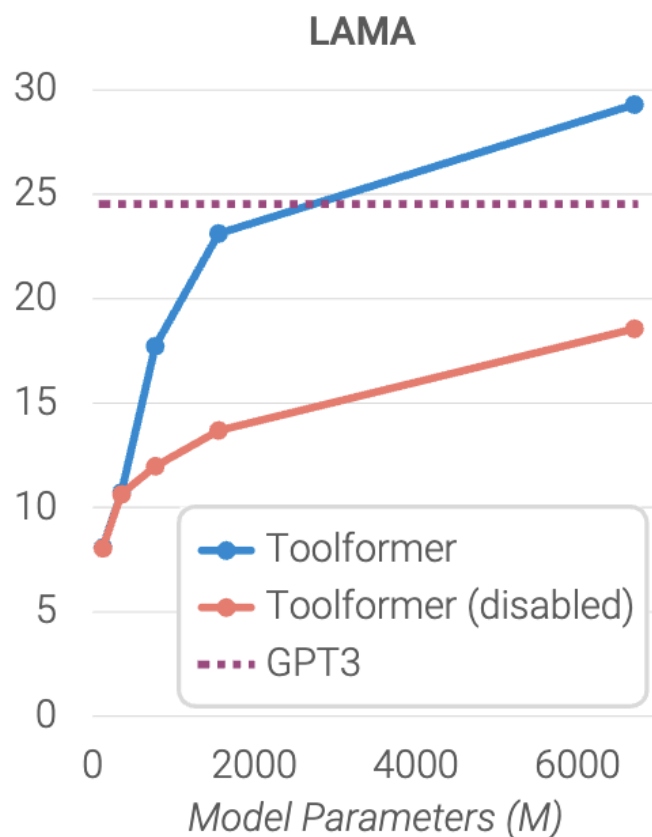
-1.23

The last time I was with [QA("Who was the last time I was with?") → The Last Time] him I asked what he likes about me.

Toolformer Result on Diverse Tasks

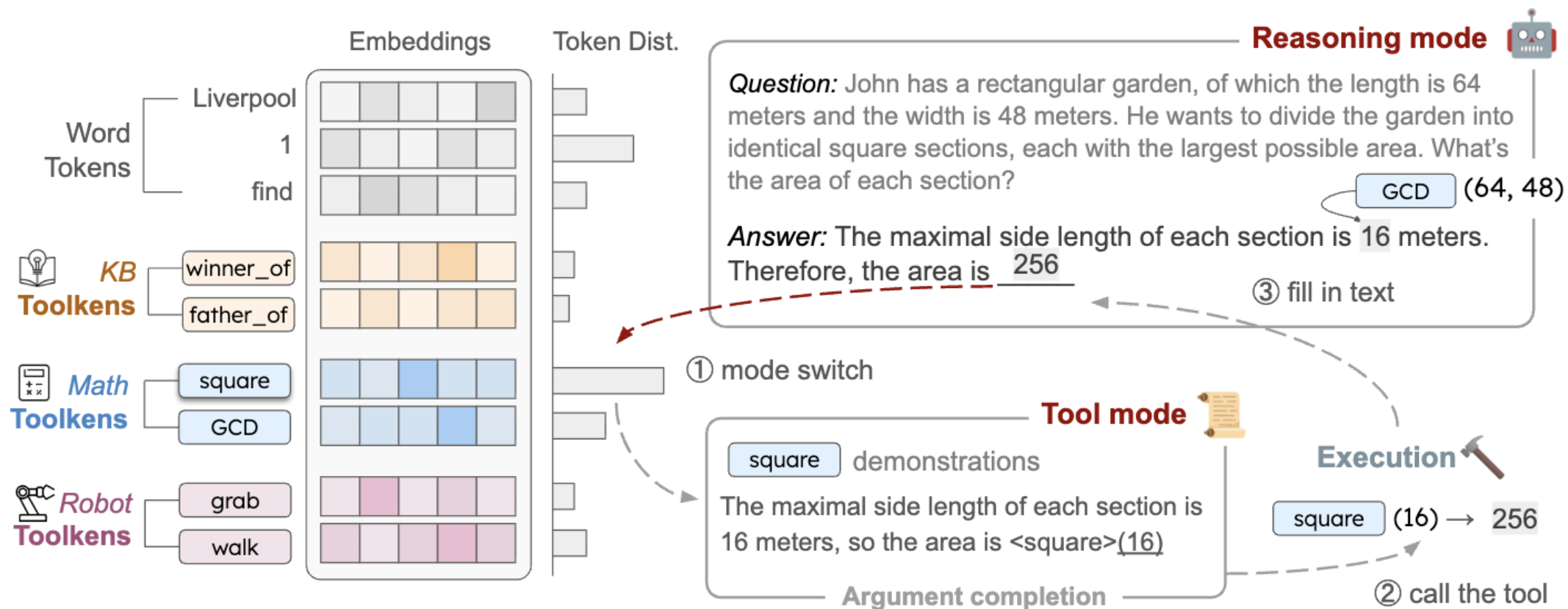


GPT-2 Models with and without Toolformer



View Tools as “Words”? Toolken GPT [\[Hao et al., 2024\]](#)

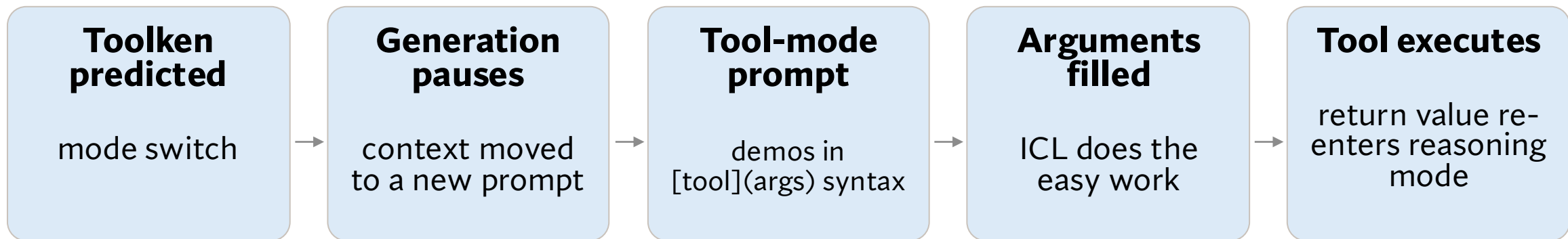
- Represent each tool as a token (“toolken”) and learns an embedding for it.
- Once a toolken is triggered, LM will complete arguments for the tool to execute.



ToolkenGPT Framework

- LLM predicts the next token (either a word or a toolken)

$$P(t_i|t_{<i}) = \text{softmax}([W_\nu; W_\tau] \cdot h_{i-1})$$



Learning Toolken Embeddings

“the area is 256 square feet”

s	the	area	is	2	5	6	square	feet
s'	the	area	is	[square]	[N/A]	[N/A]	square	feet

[square]: toolken, predicted at the first result token

[N/A]: the result tokens, skipped in the loss

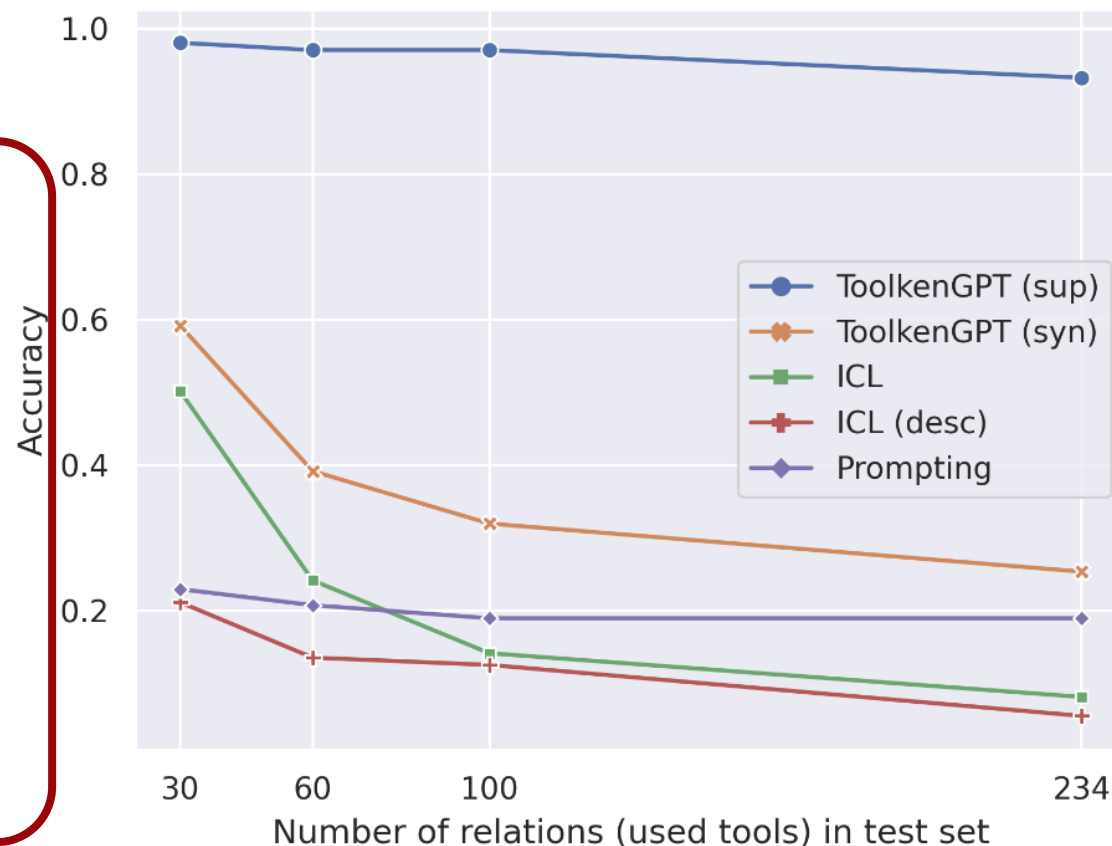
loss only at toolken positions

$$\mathcal{L}(W_{\tau}) = \sum_{(s,s') \in \mathcal{D}} \sum_{i=1 \dots N} -\log P(t'_i | t_{<i}) \cdot \mathbb{1}[t'_i \neq \text{N/A}]$$

Training data: dataset with groundtruth tool calls; synthetic data

ToolkenGPT involving different numbers of tools

Method	GSM8K-XL (4)	FuncQA (13)	
		One-Hop	Multi-Hops
0-shot ChatGPT	0.17	0.55	0.09
CoT [65]	0.18	0.20	0.03
ReAct [69]	0.32	0.57	0.06
ToolkenGPT (Ours)	0.33	0.73	0.15

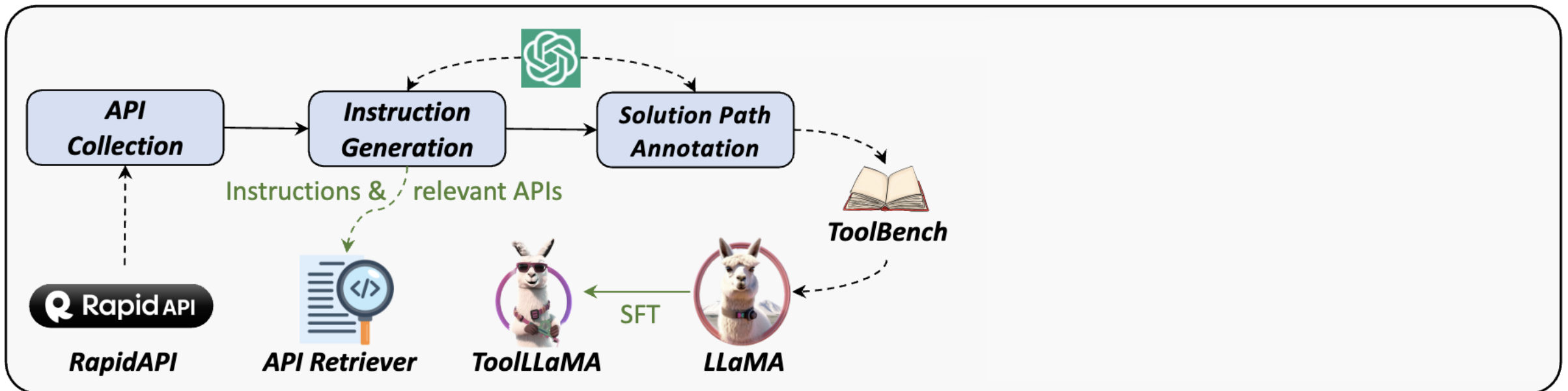


Lecture Overview

- Wrap up RAG (5 mins)
- Why tools? (5 mins)
- What are tools? (5 mins)
- Learning to call tools: Toolformer (15 mins)
- Scaling to many tools: Gorilla (10 mins)
- Standard interfaces: REST, function calling, MCP (15 mins)
- Evaluating tool use: BFCL, ToolBench, τ -bench (15 mins)

ToolLLM: Enable LLMs to Use 16k+ APIs

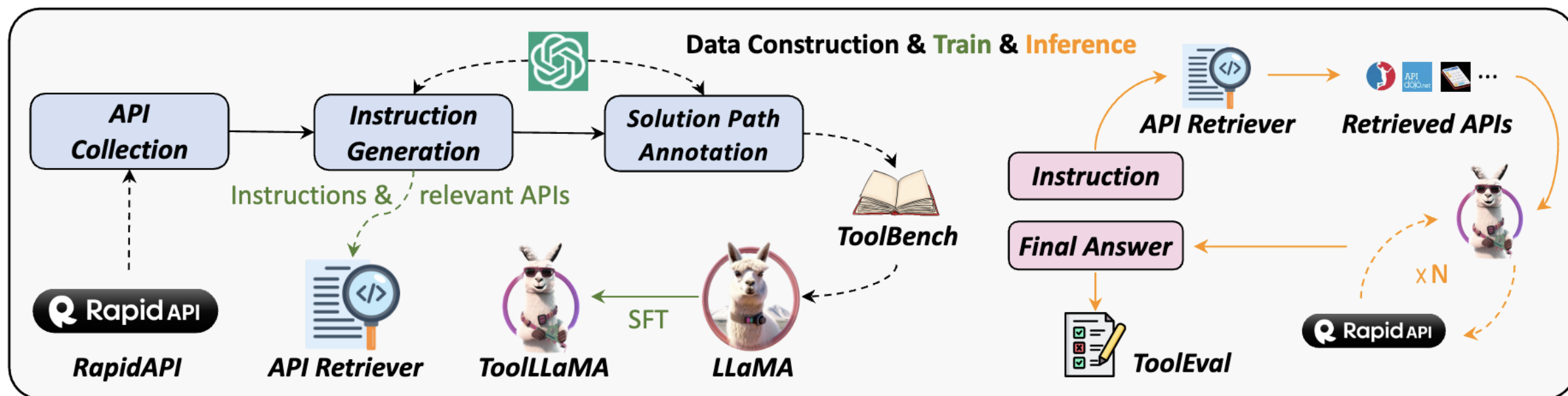
Fine-tune LLaMA to obtain an LLM ToolLLaMA, and equip it with a neural API retriever to recommend appropriate APIs for each instruction



<https://github.com/OpenBMB/ToolBench>

ToolLLM: Enable LLMs to Use 16k+ APIs

Fine-tune LLaMA to obtain an LLM ToolLLaMA, and equip it with a neural API retriever to recommend appropriate APIs for each instruction



<https://github.com/OpenBMB/ToolBench>

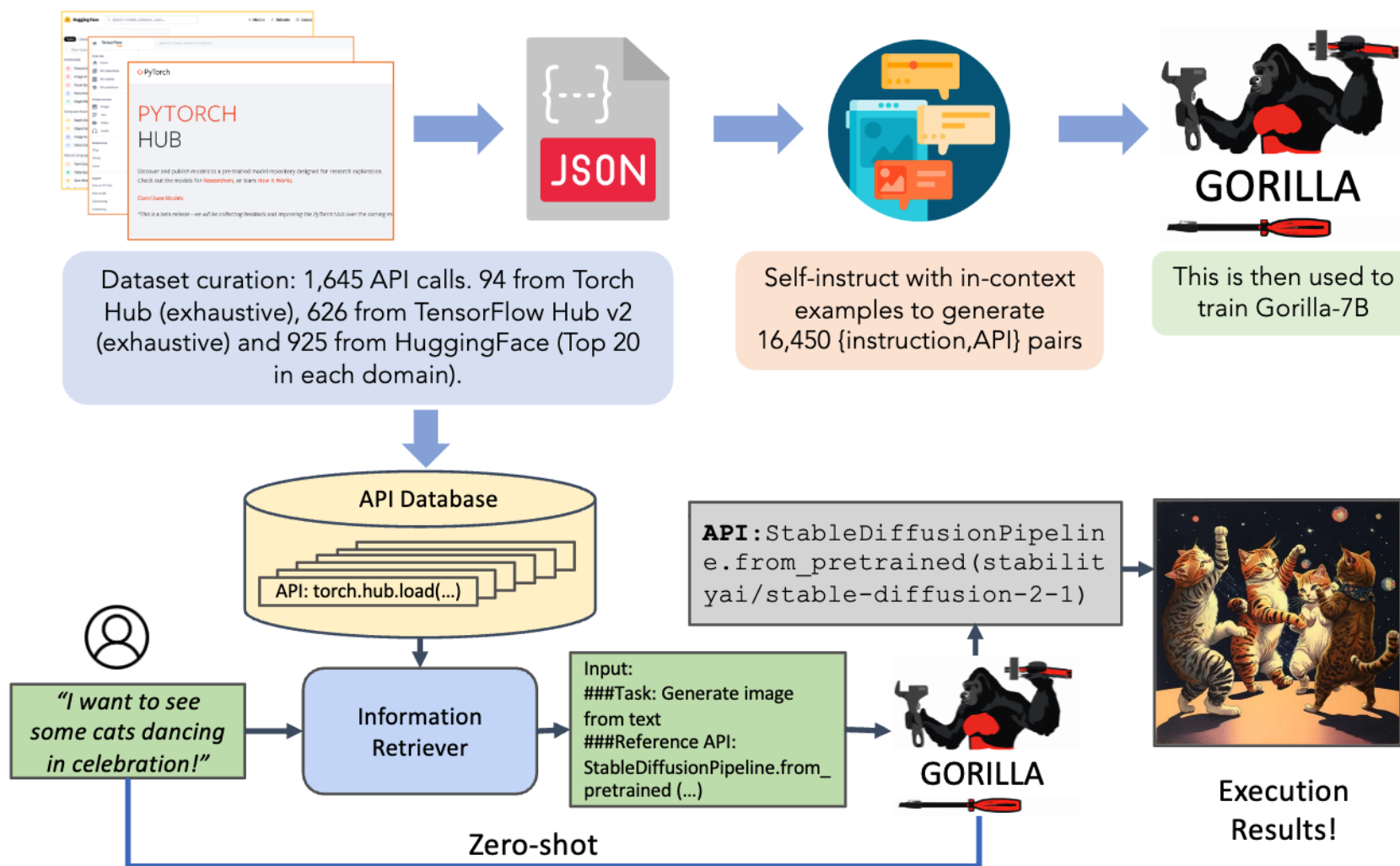
Arising Issue in Tool Use and Hallucination

GPT-4	Claude	Gorilla
<pre><domain>: Speech-to-Text <api_provider>: TorchHub <code>: asr_model = torch.hub.load('snakers4/silero-models', 'asr', source='local') result = asr_model.transcribe(audio_path)</pre>	<pre><domain>: Audio-Translation <api_provider>: Pytorch <code>: import torchaudio translation = Torchaudio.pipelines. WAV2VEC2_ASR_PIPELINE("audio.wav")</pre>	<pre><domain>: Speech-to-Text <api_provider>: TorchHub <code>: asr_model = torch.hub.load('snakers4/silero-models', 'silero_sst') result = asr_model.transcribe(audio_path)</pre>
 Hallucinate!	 Wrong library!	 Good to go!

Prompt: Help me find an API to convert the spoken language in a recorded audio to text using Torch Hub.

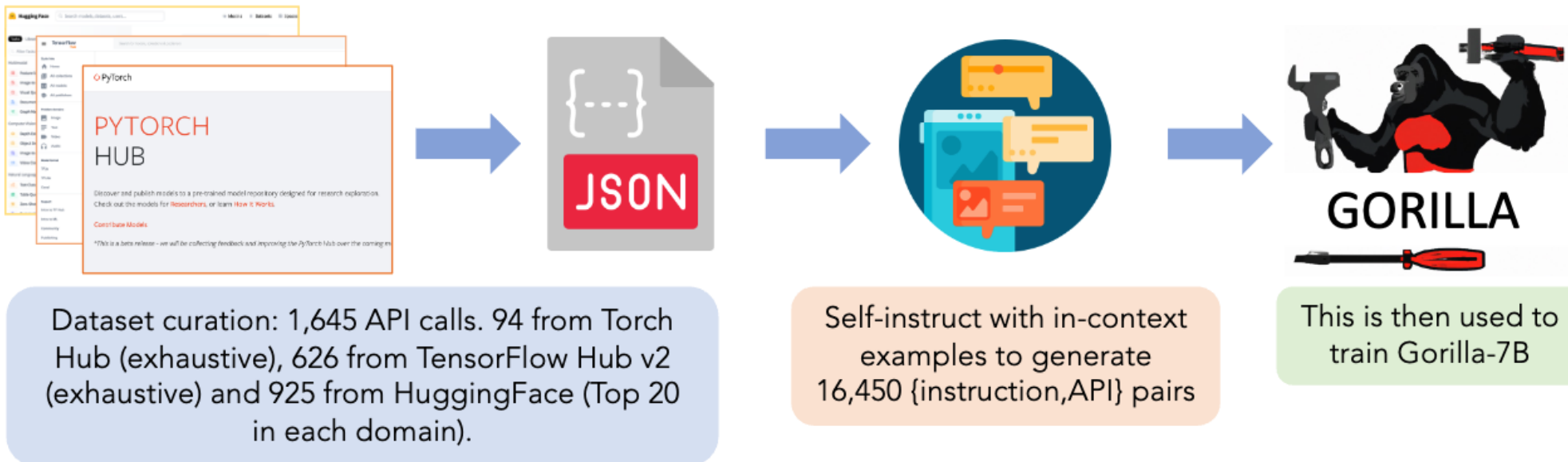
Gorilla: LLMs connected with massive APIs

(Patil et al., 2023)



Data: Scaling to Many Tools

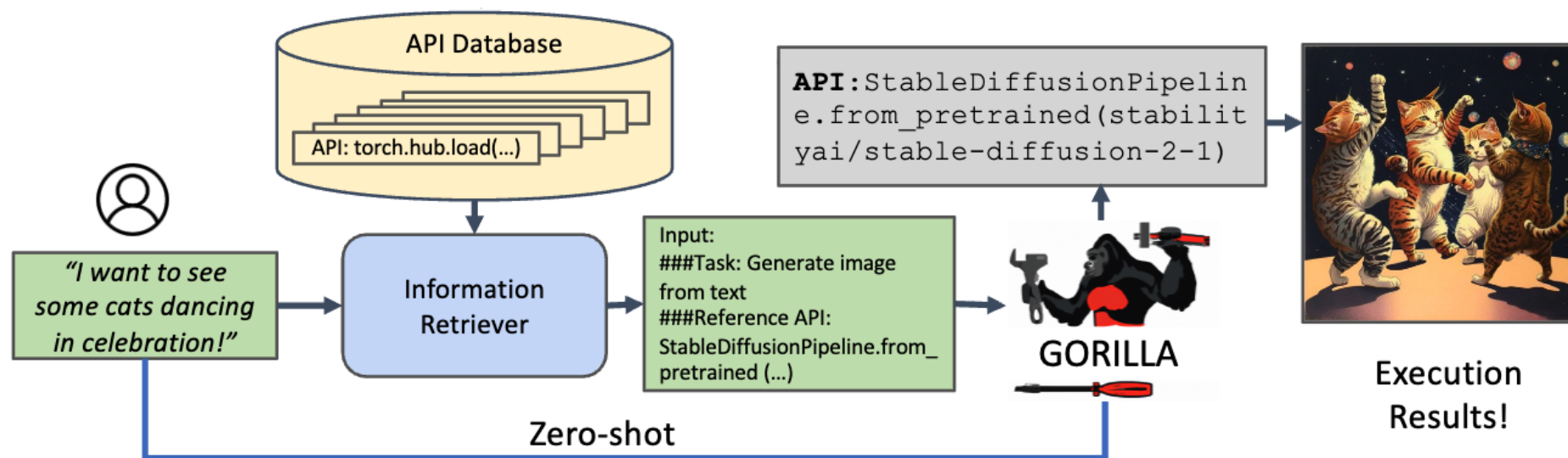
- APIs: HuggingFace’s “The Model Hub”, PyTorch Hub, and TensorFlow Hub
- 1645 API calls in json format {domain, framework, functionality, api_name, api_call, api_arguments, environment_requirements, example_code, performance, description}
- Use GPT-4 to generate synthetic instruction data



Retrieval Aware Training

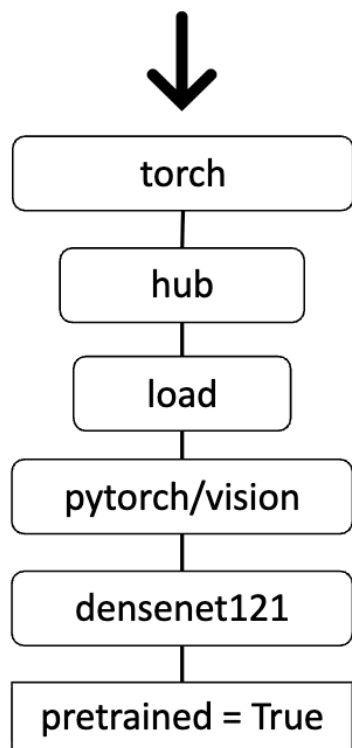
- Fine-tune the model to use or ignore retrieval context
- Introduce correct and incorrect retrieval during instruction fine-tuning
- RAT ensures the model is robust to low-quality retrieval

Gorilla
Inference:
zero-shot
or retrieval

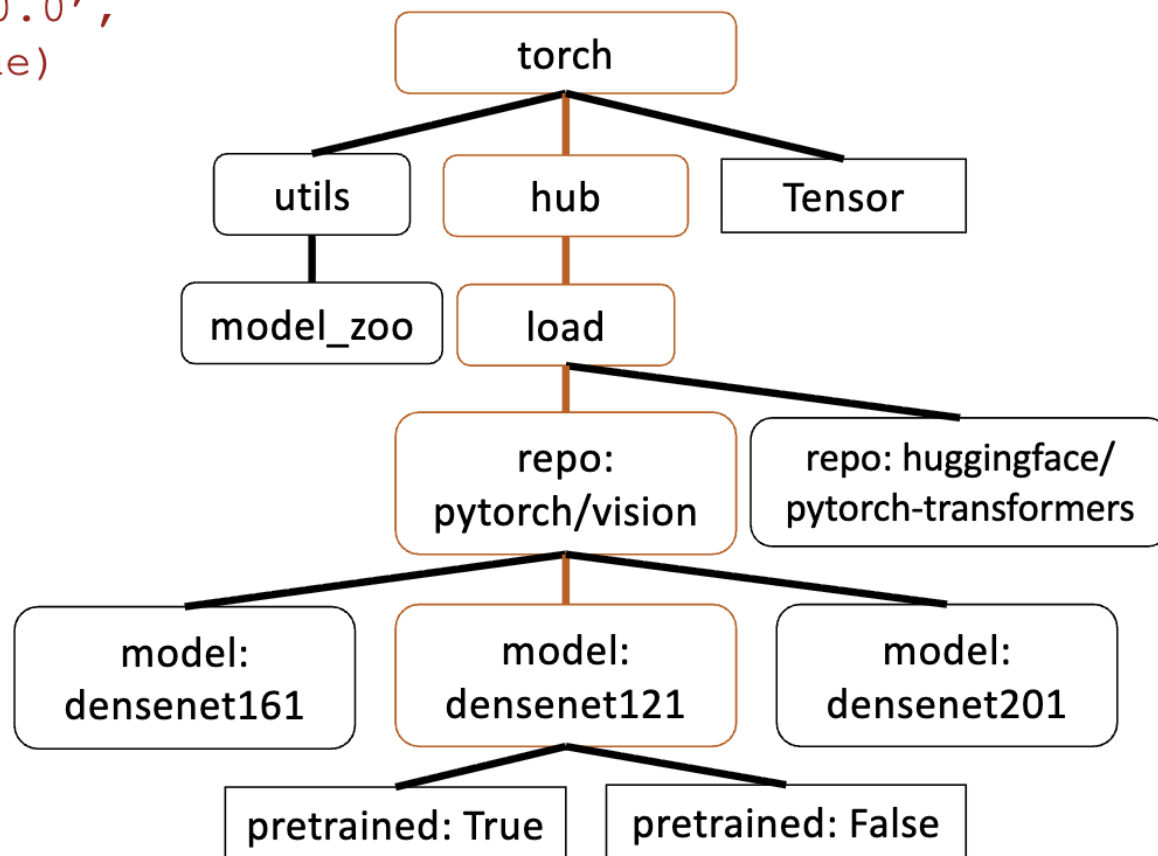


AST Sub-Tree Matching to measure hallucination

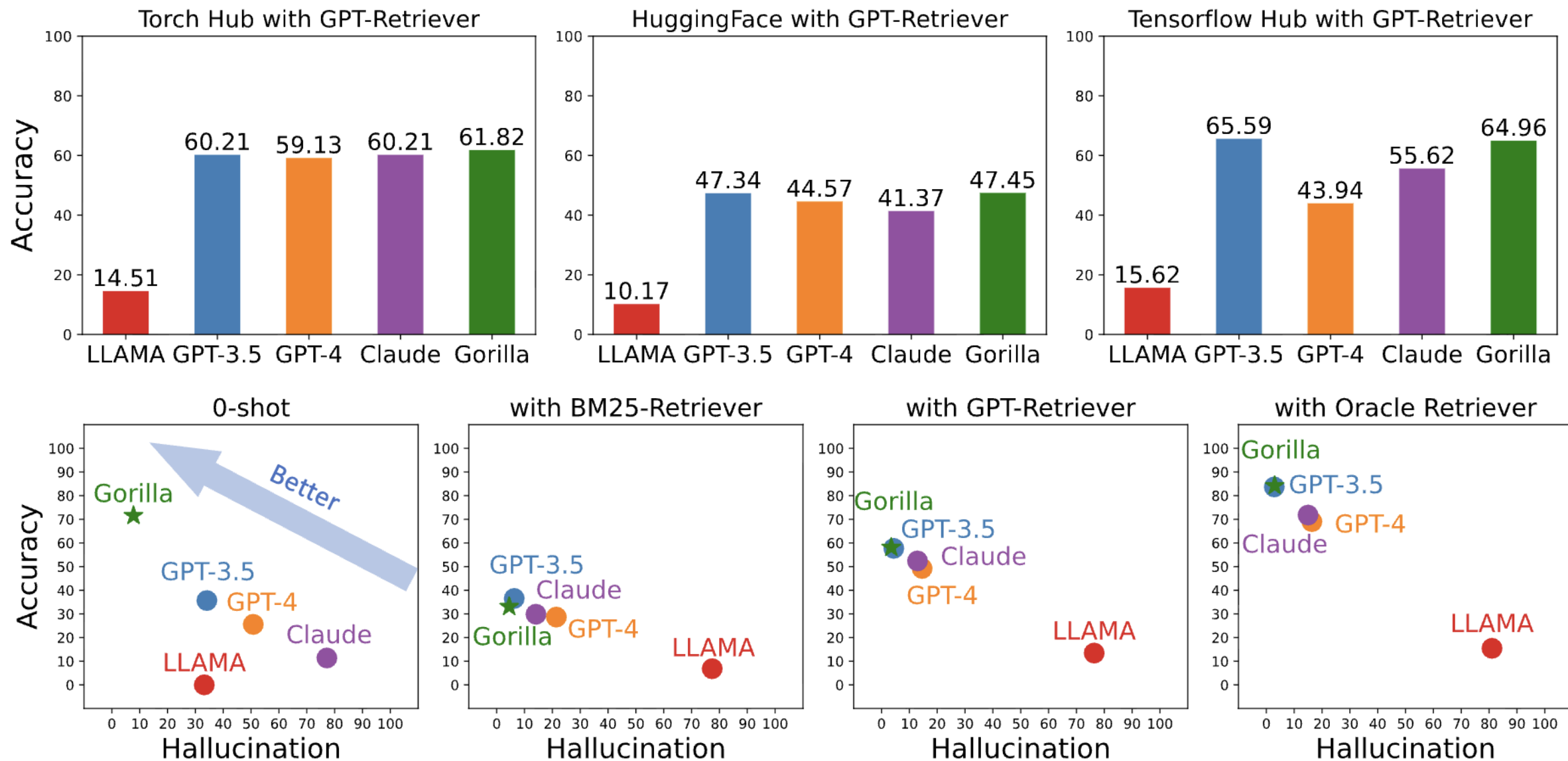
```
torch.hub.load('pytorch/vision:v0.10.0',  
              'densenet121', pretrained=True)
```



∈



Improved accuracy with reduced hallucination



Learning to Use Tools: Summary

	Toolformer	ToolkenGPT	ToolLLM	Gorilla
Tool call is	text	one new token	ReAct-style trace	text, API call code
What is trained	model (GPT-J 6B)	embeddings	model (LLaMA 7B)	model (LLaMA 7B)
Labels	self-supervised	ground-truth or synthetic calls	ChatGPT traces	GPT-4 self-instruct
Learns when	loss filter	embedding	fine-tuning	fine-tuning
Learns what	fine-tuning	in-context	fine-tuning	fine-tuning
Scale	5 tools	234 tools	16k APIs	1.6k APIs

Lecture Overview

- Wrap up RAG (5 mins)
- Why tools? (5 mins)
- What are tools? (5 mins)
- Learning to call tools: Toolformer (15 mins)
- Scaling to many tools: Gorilla (10 mins)
- Standard interfaces: REST, function calling, MCP (15 mins)
- Evaluating tool use: BFCL, ToolBench, τ -bench (15 mins)

Why Standard Interfaces?

Change the **model** to fit the tools

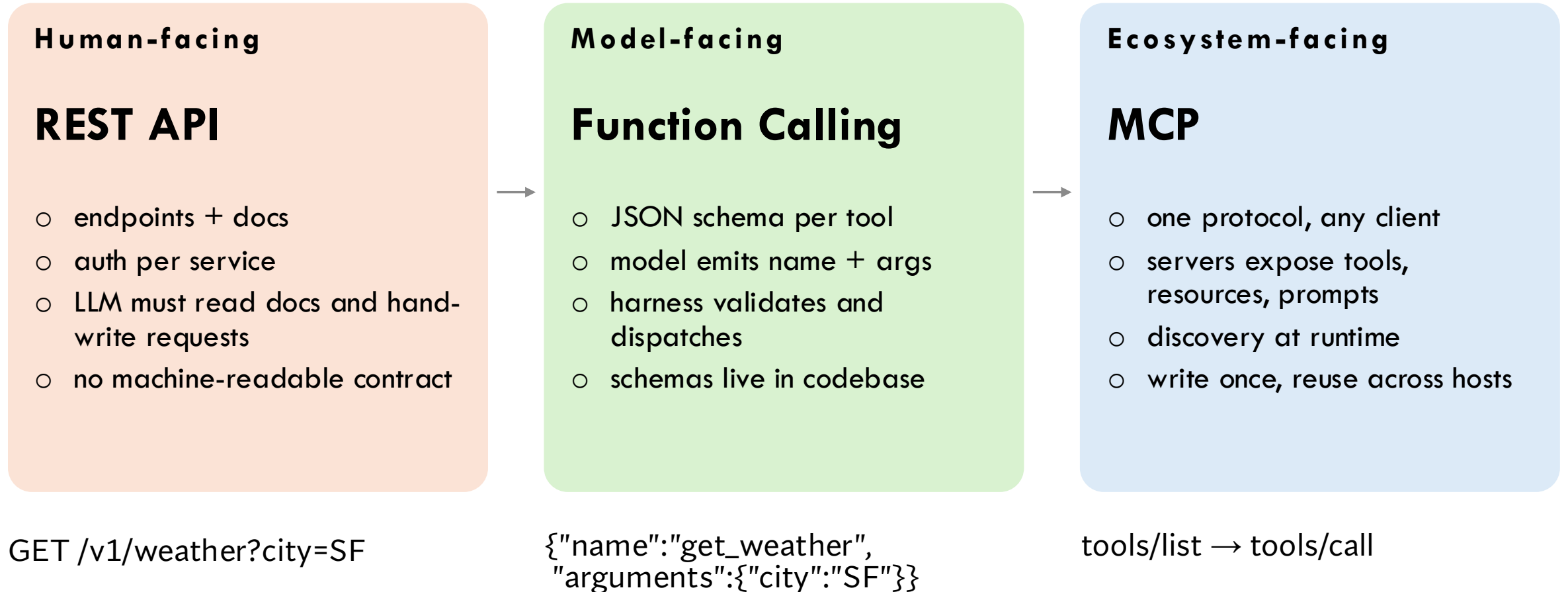
Toolformer, ToolkenGPT, Gorilla:
every new API means another training



Change how the **tools** are described so any model can use them

REST, function calling, MCP: a schema at runtime instead of fine-tune per API

Standard Interfaces: Three Generations



REST APIs: Tools Over HTTP

```
GET https://api.weather.com/v1/current?city=Stanford
```

```
200 {"temp_f": 74, "conditions": "sunny"}
```

GET read **POST** create **PUT** update **DELETE** remove

Limitation: the model must know the URL, parameter names and response shape

Function Calling: Tools as JSON Schemas

Tool Schema

```
{ "name": "get_weather",  
  "description": "Current weather",  
  "parameters": {  
    "type": "object",  
    "properties": {  
      "city": {"type": "string"} },  
    "required": ["city"] } }
```



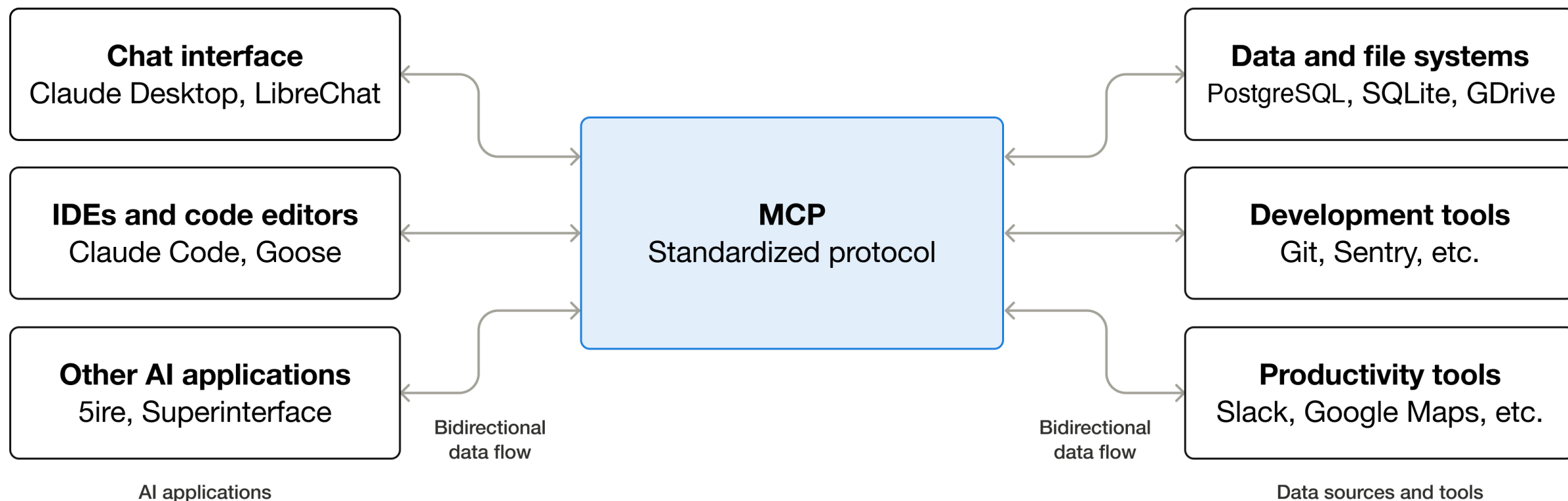
Model Output

```
{ "tool_call": {  
  "name": "get_weather",  
  "arguments": {"city": "Stanford"} } }
```

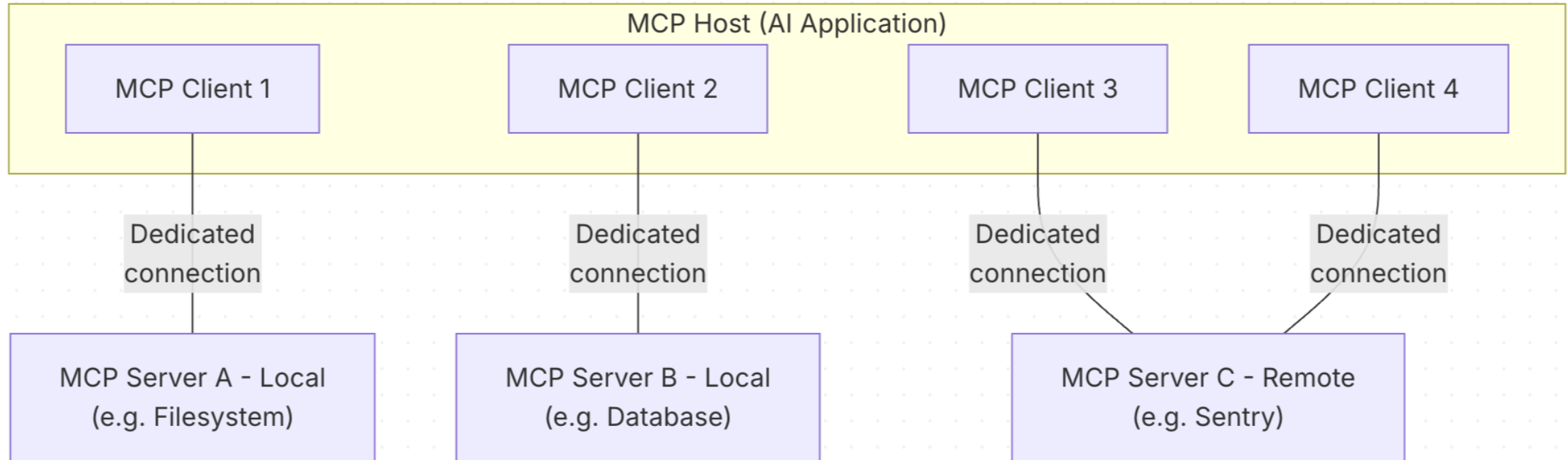
schema tool inputs **tool_call** model output **result** back to the model

Limitation: a tool written for one API has to be rewritten for the next

Model Context Protocol (MCP)



MCP Architecture: MCP Host, MCP Client, MCP Server



MCP: Tool Discovery (tools/list)

REQUEST

```
{ "jsonrpc": "2.0", "id": 2,  
  "method": "tools/list",  
  "params": { "_meta": { ... } } }
```

RESPONSE

```
{ "id": 2, "result": { "tools": [  
  { "name": "weather_current",  
    "title": "Weather Information",  
    "description": "Current weather for a  
location",  
    "inputSchema": { "type": "object",  
      "properties": {  
        "location": { "type": "string"},  
        "units": { "enum": ["metric",  
"imperial"]} },  
      "required": ["location"] } },  
  { "name": "calculator_arithmetic", ... } ],  
  "ttlMs": 300000 } }
```

“What tools do you have?”

- Server returns a list
- Each tool has a name, description and JSON inputSchema

MCP: Tool Execution (tools/call)

REQUEST

```
{ "jsonrpc": "2.0", "id": 3,  
  "method": "tools/call",  
  "params": {  
    "name": "weather_current",  
    "arguments": {  
      "location": "Stanford",  
      "units": "imperial" },  
    "_meta": { ... } } }
```

RESPONSE

```
{ "id": 3, "result": {  
  "content": [  
    { "type": "text",  
      "text": "Current weather in Stanford:  
68°F, partly cloudy, wind W 8 mph" }  
  ] } }
```

“Run this tool with these arguments”

- **name** picks a listed tool
- **arguments** follow its inputSchema
- Result is a content array the host hands back to the LLM

Example: Course LMS API → MCP Tools

REST route	MCP tool
GET /assignments	list_assignments
GET /submissions	get_submissions
POST /grade	post_grade

```
from fastmcp import FastMCP
import httpx

api = httpx.AsyncClient(
    base_url="https://lms.stanford.edu/api/v1")

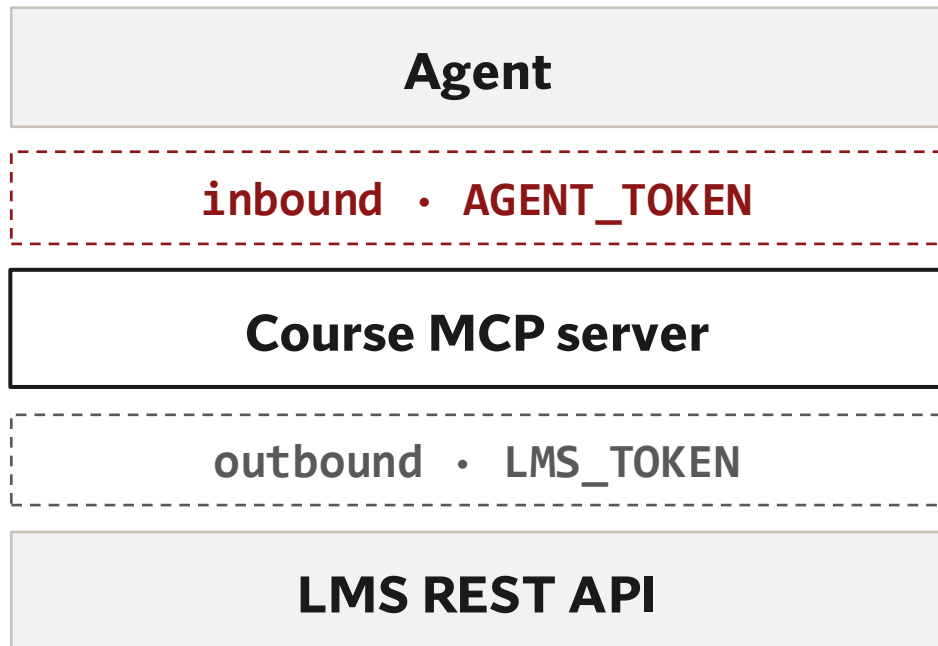
spec = (await api.get("/openapi.json")).json()

mcp = FastMCP.from_openapi(
    openapi_spec=spec,
    client=api,
    name="Course MCP",
)

# → list_assignments, get_submissions, post_grade
```

<https://gofastmcp.com/integrations/openapi>

Example: Inbound vs. Outbound Auth



```
import os, httpx
from fastmcp import FastMCP
from fastmcp.server.auth.providers.jwt import StaticTokenVerifier

# outbound: server → LMS
api = httpx.AsyncClient(
    base_url="https://lms.stanford.edu/api/v1",
    headers={"Authorization":
             f"Bearer {os.environ['LMS_TOKEN']}"})

# inbound: agent → server
who_may_call = StaticTokenVerifier(tokens={
    os.environ["AGENT_TOKEN"]: {"client_id": "ta-agent"}})

mcp = FastMCP.from_openapi(openapi_spec=spec, client=api,
                          name="Course MCP", auth=who_may_call)
mcp.run(transport="http", port=8001)
```

<https://gofastmcp.com/servers/auth/token-verification>

Lecture Overview

- Wrap up RAG (5 mins)
- Why tools? (5 mins)
- What are tools? (5 mins)
- Learning to call tools: Toolformer (15 mins)
- Scaling to many tools: Gorilla (10 mins)
- Standard interfaces: REST, function calling, MCP (15 mins)
- Evaluating tool use: BFCL, ToolBench, τ -bench (15 mins)

BFCL: Berkeley Function-Calling Leaderboard (Patil et al., 2025)

				Agentic		Multi Turn	Single Turn	
				Web Search ►	Memory ►	Multi turn ►	Non-live (AST) ►	Live (AST) ►
Rank ▲	Overall Acc	Model	Cost (\$)	Overall Acc	Overall Acc	Overall Acc	Overall Acc	Overall Acc
1	77.47	Claude-Opus-4-5-20251101 (FC)	86.55	84.5	73.76	68.38	88.58	79.79
2	73.24	Claude-Sonnet-4-5-20250929 (FC)	43.73	81	64.95	61.37	88.65	81.13
3	72.51	Gemini-3-Pro-Preview (Prompt)	298.47	80	61.72	60.75	90.65	83.12
4	72.38	GLM-4.6 (FC thinking)	4.64	77.5	55.7	68	87.56	80.9
5	69.57	Grok-4-1-fast-reasoning (FC)	17.26	82.5	53.98	58.87	88.27	78.46

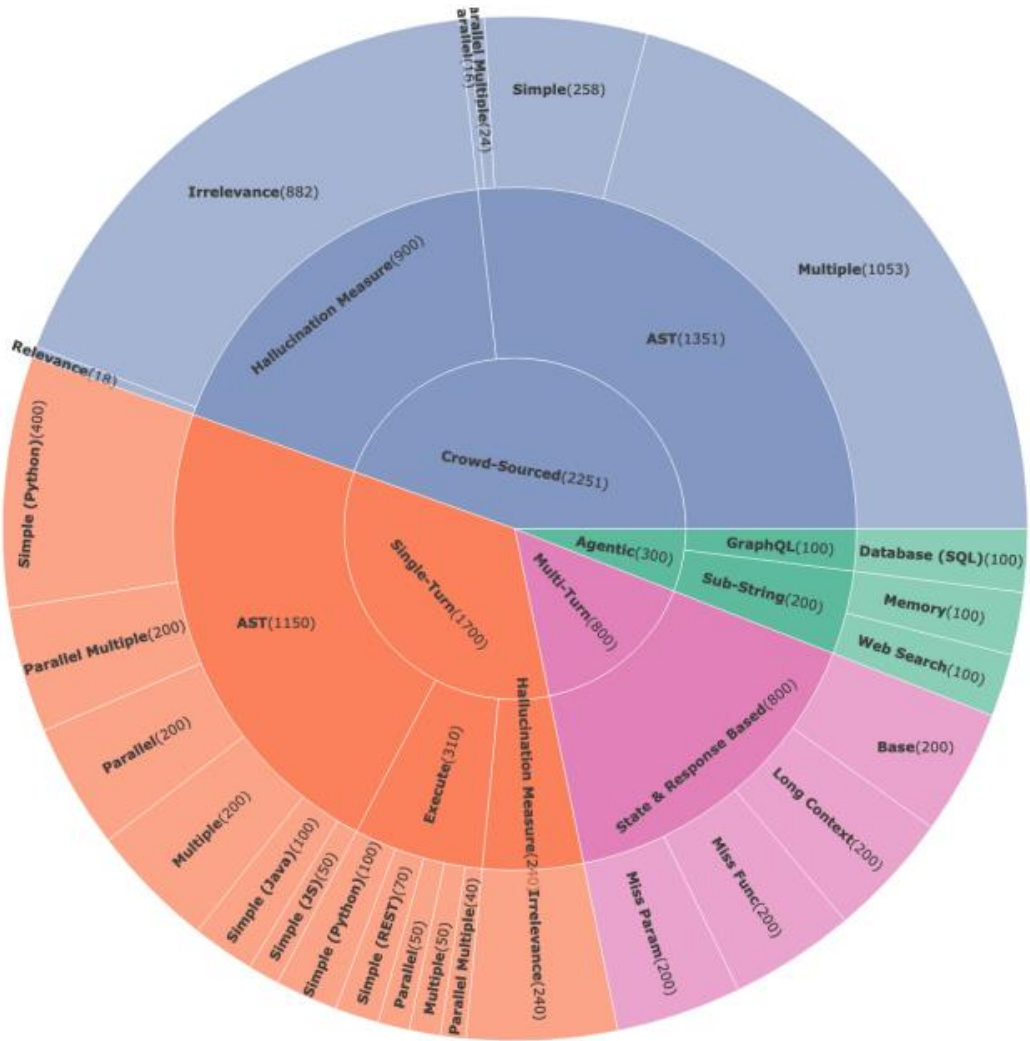
<https://gorilla.cs.berkeley.edu/leaderboard.html> Leaderboard snapshot, Nov 2025

Berkeley Function Calling Leaderboard

- **BFCL:** a comprehensive evaluation framework of a LLM's ability to invoke functions (also known as tool calling)
- **Evaluation:**
 - Abstract syntax tree (AST) based offline evaluation
 - Execution of the function for real-world evaluation
 - State based evaluation for multi-step and multi-turn function calls
- **Datasets:**
 - Sourced from real-world use-cases
 - Establish different taxonomies for function calling
 - Evaluation Python, C++, Java, SQL, ...
 - Tests relevance and irrelevance (none of the function choices provided are relevant to the user query)

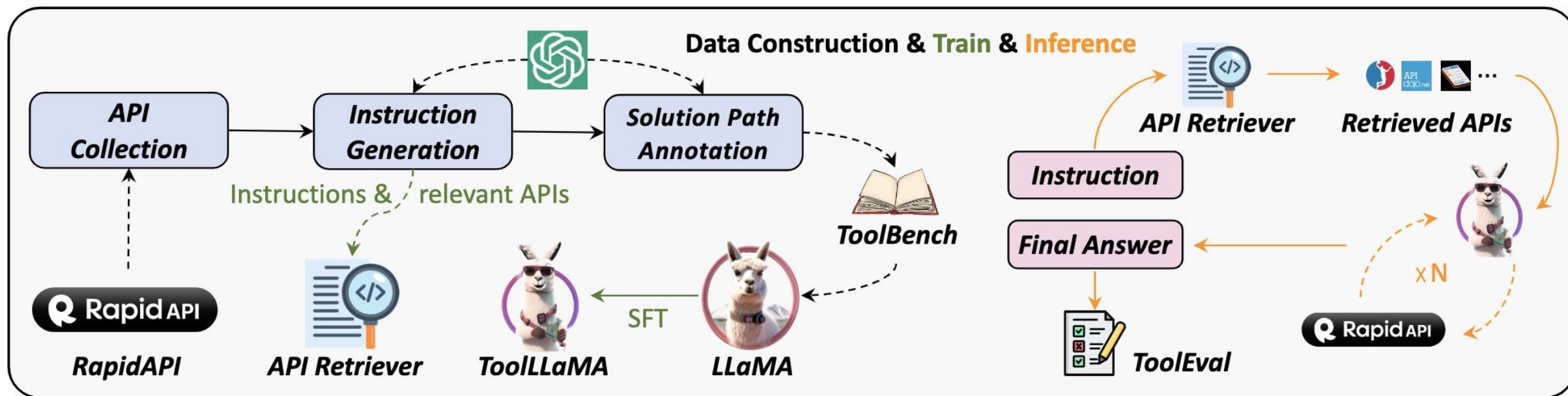
Example Function Calls

Multiple Functions	Parallel Functions	Relevance Detection
USER What is 2 + 3?	USER What is (2+3) and (4+5)?	USER What is 2*3?
FUNCTIONS add(a, b) mult(a, b)	FUNCTIONS add(a, b)	FUNCTIONS add(a, b)
AGENT add(a=2, b=3)	AGENT add(a=2, b=3), add(a=4, b=5)	AGENT Error: request is multiplication; only add is available.

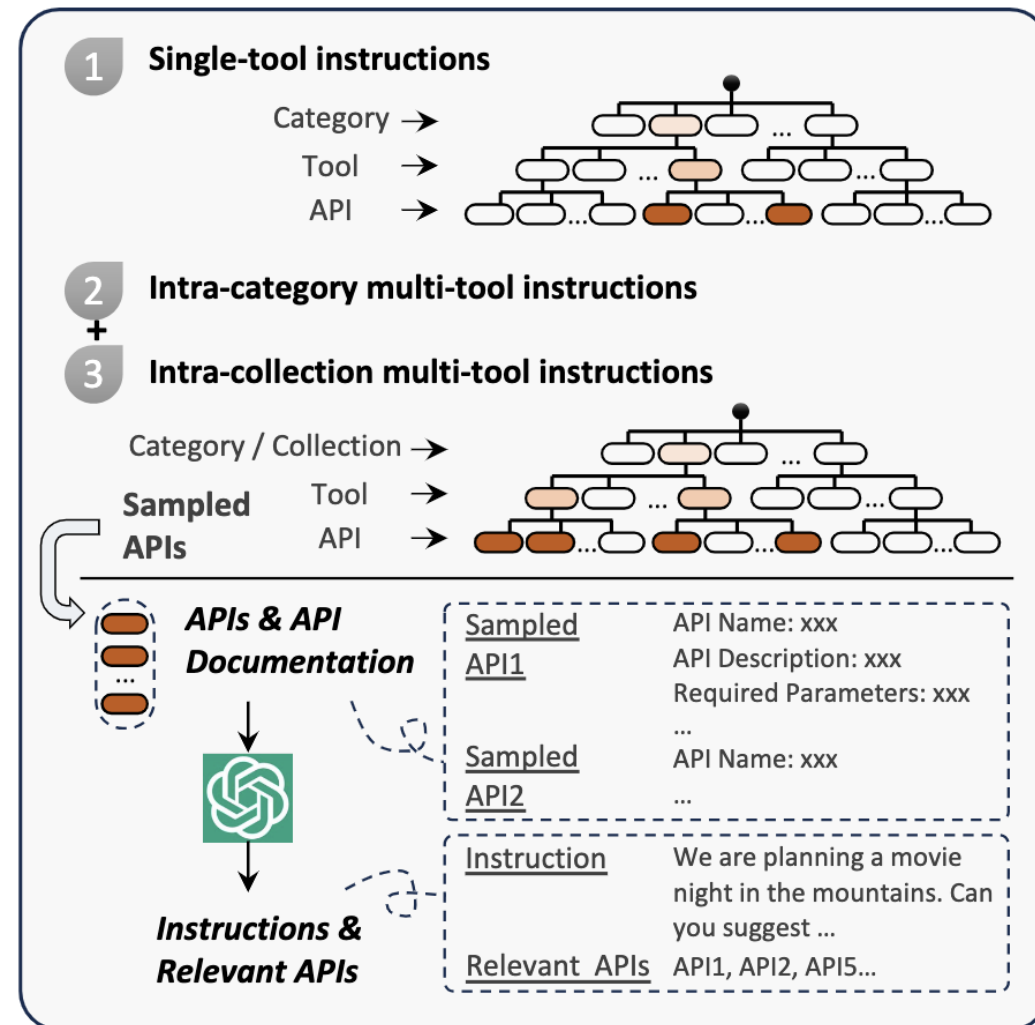
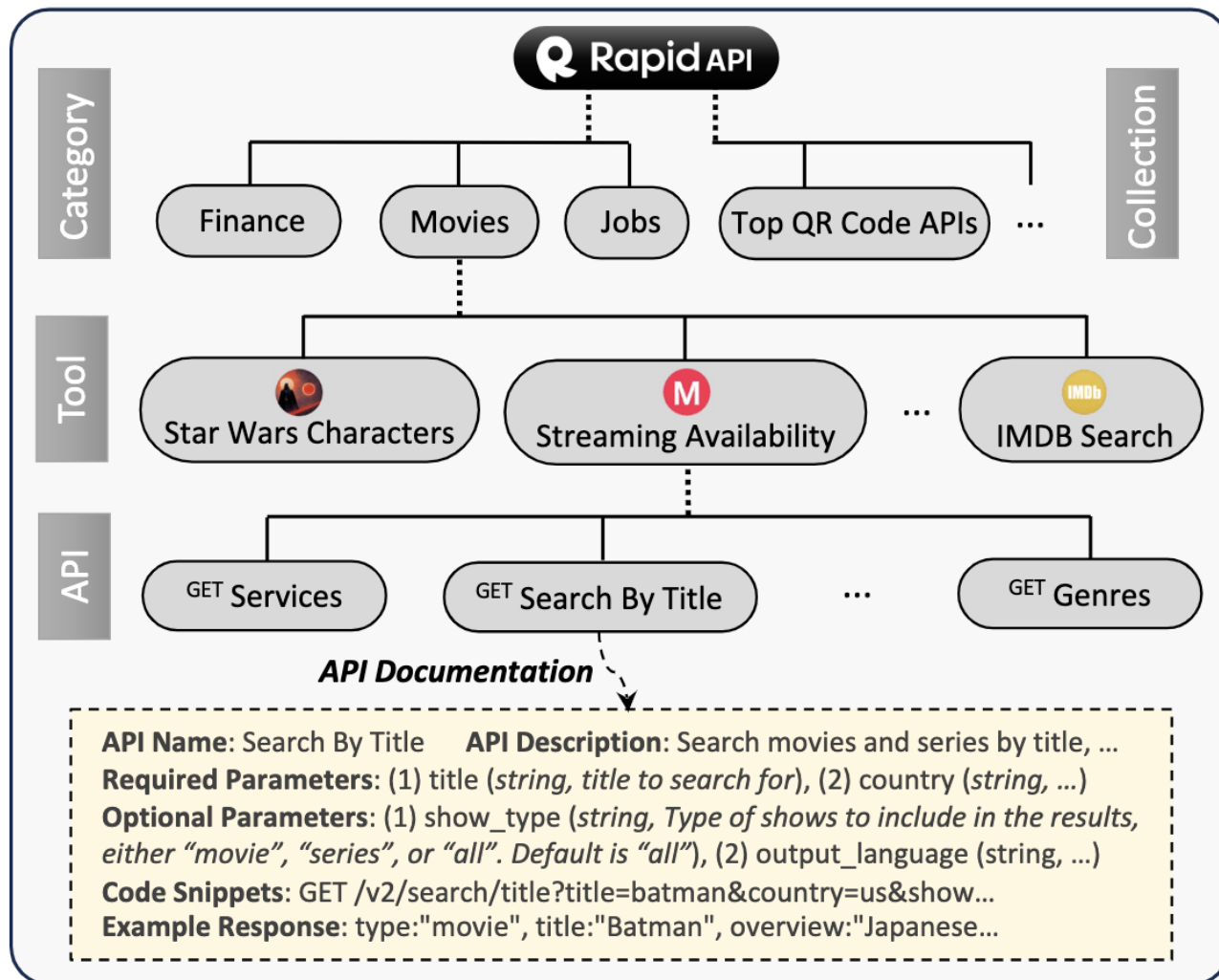


ToolBench / ToolLLM: 16k Real APIs (Qin et al., 2023)

- Crawl 6000+ real-world APIs from RapidAPI, and curate realistic human instructions that involve them

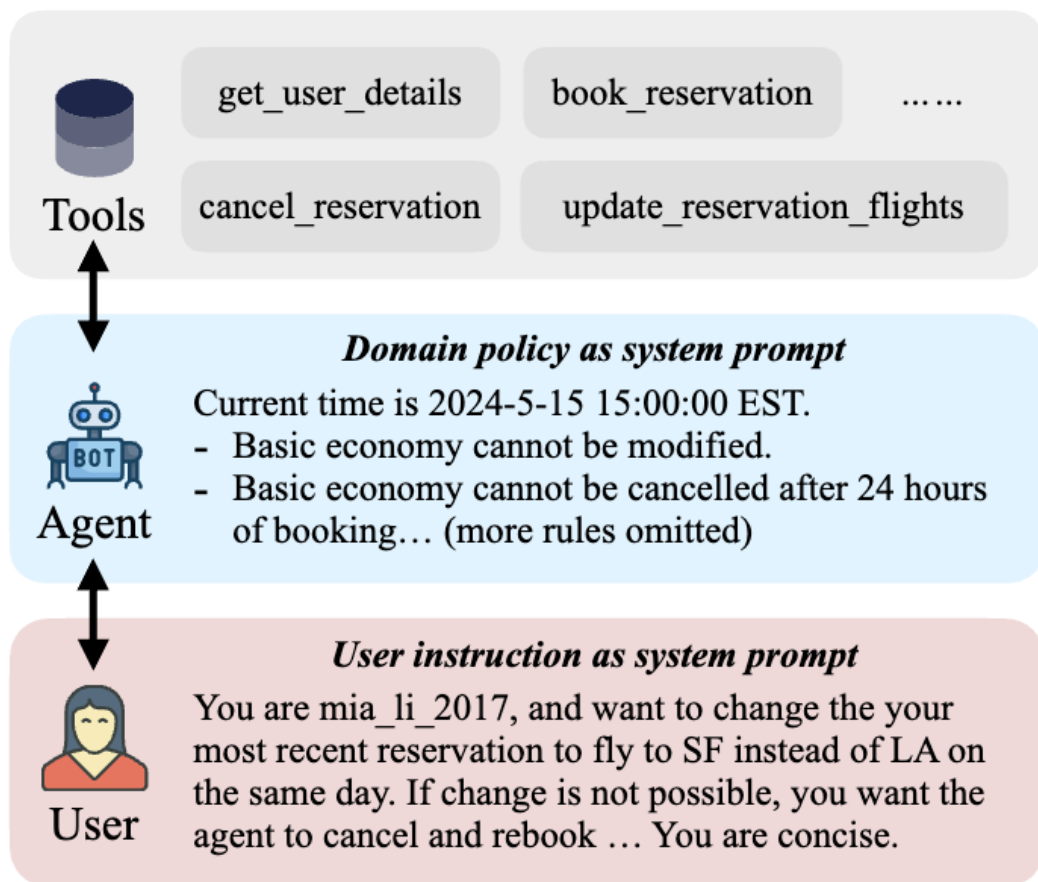


ToolBench / ToolLLM: 16k Real APIs (Qin et al., 2023)

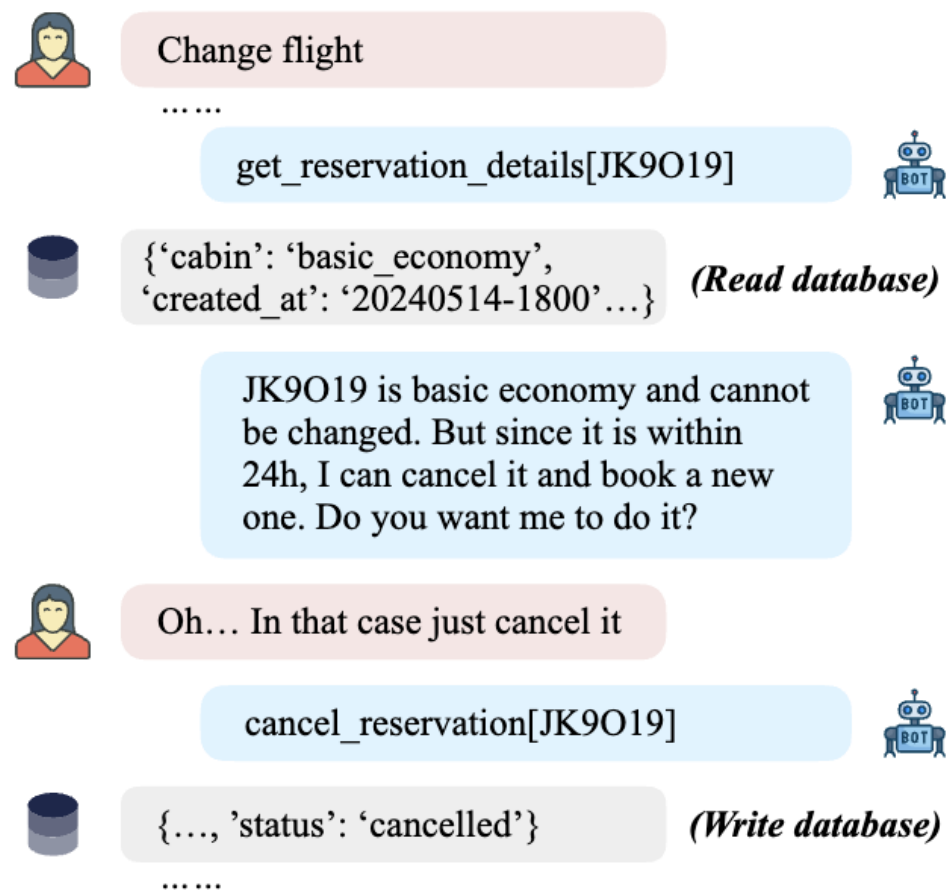


τ -bench: Tool-Agent-User Interaction (Yao et al., 2024)

(a) τ -bench setup



(b) Example trajectory in τ -airline



Pass^k to reward reliability and consistency

$$\text{pass}^k = \mathbb{E}_{\text{task}} \left[\frac{\binom{c}{k}}{\binom{n}{k}} \right], \quad \text{pass}@k = 1 - \mathbb{E}_{\text{task}} \left[\frac{\binom{n-c}{k}}{\binom{n}{k}} \right]$$

P(all k sampled trials succeed),
given c of n succeed

P(≥ 1 of k sampled trials succeeds),
given c of n succeed

	τ -retail	τ -airline
Databases	500 users, 50 products, 1,000 orders	500 users, 300 flights, 2,000 reservations
API tools	7 write, 8 non-write	6 write, 7 non-write
Tasks	115	50

Model	retail	airline	avg
gpt-4o	61.2	35.2	48.2
gpt-4-turbo	57.7	32.4	45.1
gpt-4-32k	56.5	33.0	44.8
gpt-3.5-turbo	20.0	10.8	15.4
claude-3-opus	44.2	34.7	39.5
claude-3-sonnet	26.3	27.6	27.0
claude-3-haiku	19.0	14.4	16.7
gemini-1.5-pro	21.7	14.0	17.9
gemini-1.5-flash	17.4	26.0	21.7
mistral-large	30.7	22.4	26.6
mixtral-8x22b	17.7	31.6	24.7
meta-llama-3-70B	14.8	14.4	14.6

Table 2: Pass¹ across models via function calling, except Llama-3 via text-ReAct. Average is weighted by domains, not by tasks.

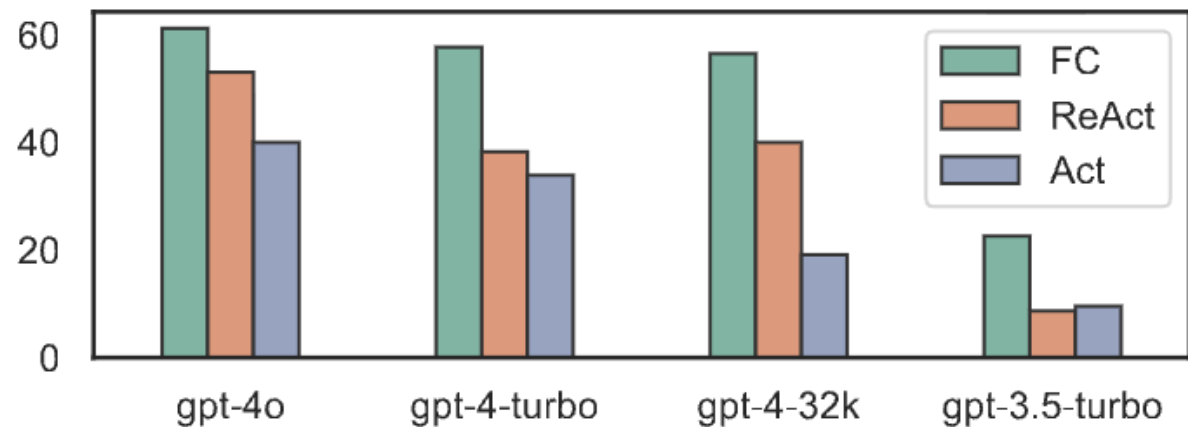


Figure 3: pass¹ across models/methods in τ -retail.

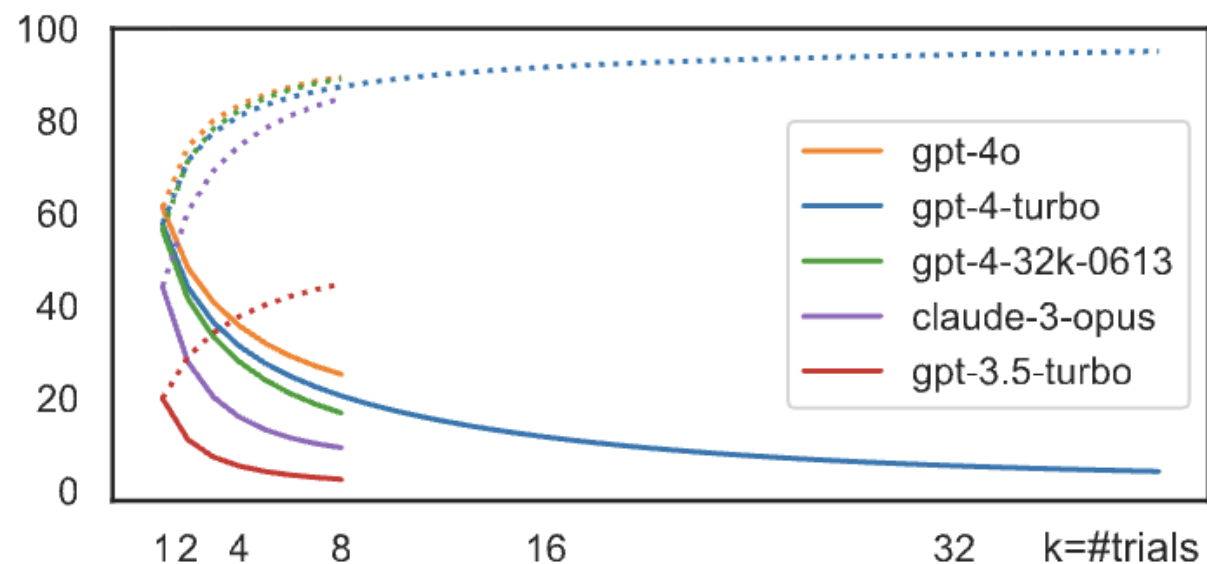


Figure 4: pass^k (—) and pass@k (..) in τ -retail.

τ -bench Has Evolved

- **τ -voice** T³-BENCH March 2026 Paper → Blog →

Real-time voice: full-duplex conversations with interruptions, accents, and background noise — the same task rigor, now in speech.

VOICE MODE:  Retail  Airline  Telecom  Banking
- **τ -knowledge** T³-BENCH March 2026 Paper → Blog →

Knowledge-intensive tasks: agents retrieve and reason over a realistic knowledge base of ~700 documents, combining retrieval with policy application.

ADDS DOMAIN:  Banking
- **Task audit & fixes** T³-BENCH February 2026 Blog →

Audited and fixed 50+ tasks across the airline and retail domains — correcting expected actions, ambiguous instructions, and impossible constraints.

UPDATES:  Retail  Airline
- **τ^2 -bench** June 2025 Paper → Blog →

Dual control: the user can now act on the world too. Agents must guide users through steps only the user can perform, not just act on their behalf.

ADDS DOMAIN:  Telecom
- **τ -bench** June 2024 Paper → Blog →

The original benchmark for tool-agent-user interaction. Agents converse with a simulated user and call tools, scored against verifiable database outcomes with the pass^k reliability metric.

DOMAINS:  Retail  Airline

Failure Modes in Tool Use

Single call

Selection: wrong tool, or a tool when none was needed

Arguments: made-up fields, wrong types or units

Hallucinated tool: calls an API that doesn't exist

Multi-step chain

Sequencing: skips a prerequisite call, wrong order

Recovery: retries the same failing call instead of backing off

Conversation

Policy: acts where the rules say not to

Interaction: assumes instead of asking the user

Non-determinism: passes once, fails on the retry (pass^k)

Lecture Summary

- ✓ **Why tools:** what LLMs didn't see in training, or can't reach
- ✓ **Toolformer:** the LM teaches itself which APIs to call, when, and with what args
- ✓ **Gorilla:** train on 1.6k API; AST matching measures hallucination
- ✓ **REST / function calling/ MCP:** from hand-written requests to tools/list, tools/call
- ✓ **BFCL / ToolBench / τ -bench:** single call, multi-step chain, tool-agent-user
- ✓ **Failure modes:** wrong tool, made-up arguments, wrong order